ADVERSARIAL STATE-SPACE OPTIMIZATION IN CHESS ENVIRONMENTS

A PROJECT REPORT

Submitted by

BAVIRI SETTY SAI DEEVAN (2116241901016)

MONISH PRIYAN M (2116241901057)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institute

CHENNAI

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report "**ADVERSARIAL STATE-SPACE OPTIMIZATION IN CHESS ENVIRONMENTS**" is the bonafide work of "**BAVIRI SETTY SAI DEEVAN (2116241901016)**" who carried out the project work under my supervision.

SIGNATURE
Dr. Bhuvana S, Ph.D.,

ASSISTANT PROFESSOR(SG)
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

Adversarial State-Space Optimization in Chess Environments is a classical artificial intelligence system that models chess as a fully observable, deterministic, zero-sum state space and plays it with a goal-based agent built from first principles, without machine learning, neural networks or external chess libraries. A self-built move generator matches the published performance test (perft) node counts on all seven standard test positions, and a hand-crafted evaluation function combines material, piece-square tables, pawn structure, king safety and mobility. Moves are chosen by Negamax search with Alpha-Beta pruning, driven by Iterative Deepening within a time budget and supported by Quiescence Search, MVV-LVA move ordering, null-move pruning, configurable contempt and a fixed-size Zobrist-hashed transposition table. The engine runs in a Web Worker behind a React interface with time controls, typed moves and a teaching mode, and is deployed on Firebase Hosting behind an automated suite of 53 tests. Across 17 positions, Alpha-Beta, move ordering and the transposition table returned exactly the Minimax move and score in all 79 comparable searches while cutting the depth-4 tree about 110-fold; null-move pruning added about 130 Elo in self-play, and the engine plays at roughly 1600–1700 on Stockfish's calibrated strength scale at one second per move. The deployed application's attack surface is analysed with STRIDE, and security controls are proposed for its delivery pipeline and hosting.

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

BAVIRI SETTY SAI DEEVAN (2116241901016)

MONISH PRIYAN M (2116241901057)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
	LIST OF TABLES	8
	LIST OF FIGURES	9
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVES	3
	1.4 SCOPE	4
	1.4.1 IN SCOPE	4
	1.4.2 OUT OF SCOPE	5
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	5
	1.6 SDG ALIGNMENT	6
2	**LITERATURE REVIEW**	**8**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	8
	2.1.1 FOUNDATIONAL RESEARCH	8
	2.1.2 EXISTING SYSTEMS	9
	2.1.3 SECURITY APPROACHES IN EXISTING SYSTEMS	9
	2.1.4 LIMITATIONS OF EXISTING SYSTEMS	11
	2.2 RESEARCH GAPS	11
	2.3 PROPOSED SYSTEM	12
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	12
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	13
	2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM	14

3 SYSTEM DESIGN 15

3.1 SYSTEM ARCHITECTURE 15

3.2 MODULE DESIGN 17

3.2.1 BOARD REPRESENTATION AND MOVE GENERATION 18

3.2.2 HEURISTIC EVALUATION FUNCTION 20

3.2.3 ADVERSARIAL SEARCH ENGINE 21

3.2.4 SEARCH OPTIMISATION 22

3.2.5 INTERFACE, TEACHING MODE AND DEPLOYMENT 24

3.2.6 VERIFICATION AND BENCHMARKING 25

3.3 DATA FLOW/UML DIAGRAMS 26

3.3.1 DATA FLOW DIAGRAMS 26

3.3.2 USE CASE DIAGRAM 28

3.3.3 ACTIVITY DIAGRAM 29

3.3.4 SEQUENCE DIAGRAM 31

3.4 THREAT MODEL AND ATTACK SURFACE 32

3.5 SECURITY CONTROLS 35

4 SYSTEM DESCRIPTION 37

4.1 DATA COLLECTION 37

4.2 DATA PREPROCESSING 38

4.2.1 ALGORITHMS / METHODOLOGY 39

4.2.1.1 PSEUDO-CODE OF THE ALGORITHMS 39

4.2.1.2 COMPLEXITY 41

4.2.1.3 EXPERIMENTAL METHODOLOGY 42

4.2.2 RESULTS AND DISCUSSION 42

4.2.2.1 CORRECTNESS 42

4.2.2.2 START POSITION AT DEPTH 5 42

4.2.2.3 ACROSS 17 POSITIONS AND DEPTHS 1–7 43

4.2.2.4 SAME ANSWER, LESS WORK 44
4.2.2.5 SPEED PER NODE 45
4.2.2.6 NULL-MOVE PRUNING 45
4.2.2.7 FIXED-SIZE TRANSPOSITION TABLE 46
4.2.2.8 CONTEMPT 46
4.2.2.9 PLAYING STRENGTH 46
4.2.2.10 WORKING PROTOTYPE 48
4.2.2.11 LIMITATIONS OBSERVED 50

5 CONCLUSION 52
5.1 SUMMARY OF THE WORK 52
5.2 FUTURE WORK 53

REFERENCES 54

APPENDIX 56
A.1 PROJECT LINKS 56
A.2 DETAILED EXPERIMENTAL RESULTS 56
A.3 REPOSITORY STRUCTURE 58
A.4 SOURCE CODE – NEGAMAX SEARCH WITH NULL MOVE (search.js) 59
A.5 SOURCE CODE – QUIESCENCE SEARCH AND MOVE ORDERING (search.js) 61
A.6 SOURCE CODE – ZOBRIST KEY COMPUTATION (transposition.js) 62
A.7 DEPLOYMENT WORKFLOW (.github/workflows/deploy.yml) 63
A.8 REPRODUCING THE RESULTS 64
A.9 SAMPLE ENGINE CONSOLE OUTPUT (RAPID GAME, ENGINE PLAYING BLACK) 64

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	TASK ENVIRONMENT PROPERTIES OF CHESS	1
1.2	SECURITY REQUIREMENTS	6
1.3	SDG ALIGNMENT	7
2.1	COMPARISON OF EXISTING SYSTEMS	10
3.1	HARDWARE REQUIREMENTS	16
3.2	SOFTWARE REQUIREMENTS	17
3.3	MODULE-TO-FILE MAPPING	17
3.4	STRIDE THREAT ANALYSIS	33
3.5	SECURITY CONTROLS	35
4.1	PERFORMANCE TEST (PERFT) REFERENCE VALUES FOR THE STANDARD TEST POSITIONS	38
4.2	EVALUATION FUNCTION TERMS	39
4.3	SEARCH CONFIGURATION COMPARISON (STARTING POSITION, DEPTH 5)	42
4.4	NODE REDUCTION RELATIVE TO PLAIN MINIMAX (GEOMETRIC MEAN)	43
4.5	AGREEMENT WITH PLAIN MINIMAX (79 SEARCHES)	44
4.6	SELF-PLAY MATCHES (80 GAMES EACH, 300 MS PER MOVE)	45
4.7	RESULTS AGAINST STOCKFISH 19 AT CALIBRATED STRENGTHS (1 S PER MOVE)	47
A.1	ENGINE PARAMETERS	56
A.2	AUTOMATED TEST SUITE	57
A.3	EFFECTIVE BRANCHING FACTOR	57
A.4	TACTICAL SUITE RESULTS (POSITIONS SOLVED OUT OF 105)	57
A.5	TRANSPOSITION-TABLE SIZE AGAINST SEARCH EFFORT (NODES AT FIXED DEPTH)	58
A.6	REPOSITORY STRUCTURE	58

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	MINIMAX GAME TREE WITH ALPHA-BETA PRUNING	2
2.1	TYPICAL FLOW OF A BASIC COURSE-LEVEL ENGINE	9
2.2	SEARCH FLOW OF THE PROPOSED SYSTEM	13
3.1	SYSTEM ARCHITECTURE	15
3.2	LEVEL-2 DFD OF MODULE 1 – BOARD REPRESENTATION AND MOVE GENERATION	19
3.3	LEVEL-2 DFD OF MODULE 2 – HEURISTIC EVALUATION FUNCTION	20
3.4	LEVEL-2 DFD OF MODULE 3 – ADVERSARIAL SEARCH ENGINE	22
3.5	LEVEL-2 DFD OF MODULE 4 – SEARCH OPTIMISATION	23
3.6	LEVEL-2 DFD OF MODULE 5 – INTERFACE, TEACHING MODE AND DEPLOYMENT	24
3.7	LEVEL-2 DFD OF MODULE 6 – VERIFICATION AND BENCHMARKING	25
3.8	DATA FLOW DIAGRAM – LEVEL 0 (CONTEXT DIAGRAM)	27
3.9	DATA FLOW DIAGRAM – LEVEL 1	28
3.10	USE CASE DIAGRAM	29
3.11	ACTIVITY DIAGRAM	30
3.12	SEQUENCE DIAGRAM	31
3.13	ATTACK SURFACE AND TRUST BOUNDARIES	32
4.1	NODES SEARCHED BY CONFIGURATION ON THE STARTING POSITION	43
4.2	NODE REDUCTION BY DEPTH (GEOMETRIC MEAN OVER 17 POSITIONS)	44
4.3	SCORE AGAINST EACH STOCKFISH SETTING VERSUS THE SCORE EXPECTED AT 1697	47
4.4	PROTOTYPE – PLAY SCREEN WITH CLOCKS AND SAN HISTORY	48
4.5	PROTOTYPE – TEACHING MODE WITH FEN PRESETS	49
4.6	PROTOTYPE – NEW-GAME DIALOG	50

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Game playing has served as a proving ground for artificial intelligence since the field began. In 1950 Claude Shannon described how a computer could play chess by searching a tree of future positions and scoring the positions at its leaves with an evaluation function [1]. Almost five decades later, IBM's Deep Blue defeated the reigning world champion Garry Kasparov using massively parallel Alpha-Beta search on custom hardware [5]. More recent engines have moved towards learned evaluation: AlphaZero learned chess entirely through self-play reinforcement learning, with a deep neural network guiding Monte-Carlo Tree Search [8], and Stockfish, the leading open-source engine, now combines classical Alpha-Beta search with an efficiently updatable neural network (NNUE) for evaluation [9], [10].

This project returns to the classical foundation. Chess is modelled as a fully observable, deterministic, two-player, zero-sum game, and a goal-based agent chooses its moves purely by searching the state space and applying a hand-crafted heuristic — without machine learning, neural networks or external datasets. Following Russell and Norvig [7], the game is defined formally by: an initial state S_0 (the standard starting position); PLAYER(s), the side to move in state s; ACTIONS(s), the set of legal moves; RESULT(s, a), the transition model that applies move a; TERMINAL-TEST(s), which detects checkmate and draws; and UTILITY(s, p), the final value of a terminal state for player p. Table 1.1 summarises the properties of the task environment that make chess suitable for this approach.

Table 1.1 Task Environment Properties of Chess

Property	Chess	Consequence for the agent
Observability	Fully observable	The complete board state is known; no belief states are needed.
Agents	Multi-agent, competitive (zero-sum)	The opponent is modelled as a rational adversary who minimises the agent's utility (Minimax).
Determinism	Deterministic	RESULT(s, a) always yields the same successor; there are no chance nodes.
Episodes	Sequential	Every move constrains all later moves, so look-ahead search is required.

Property	Chess	Consequence for the agent
Dynamics	Static (semi-dynamic with a clock)	The board does not change while the agent thinks, but the time budget matters.
States and actions	Discrete	Positions and moves are finite and can be enumerated exactly.

Because an opponent moves between the agent's turns, the agent cannot simply search for a path to a goal; it must choose the move that is best against every reply. The Minimax algorithm formalises this: MAX nodes take the largest value among their children, MIN nodes the smallest, and values are backed up from the leaves to the root. Alpha-Beta pruning computes exactly the same value while skipping branches that cannot influence the decision [3]. In Figure 1.1, once the second MIN node finds a reply worth 2, that node can be worth at most 2 to MAX, who already has a move worth 3 — so its remaining children are never examined.

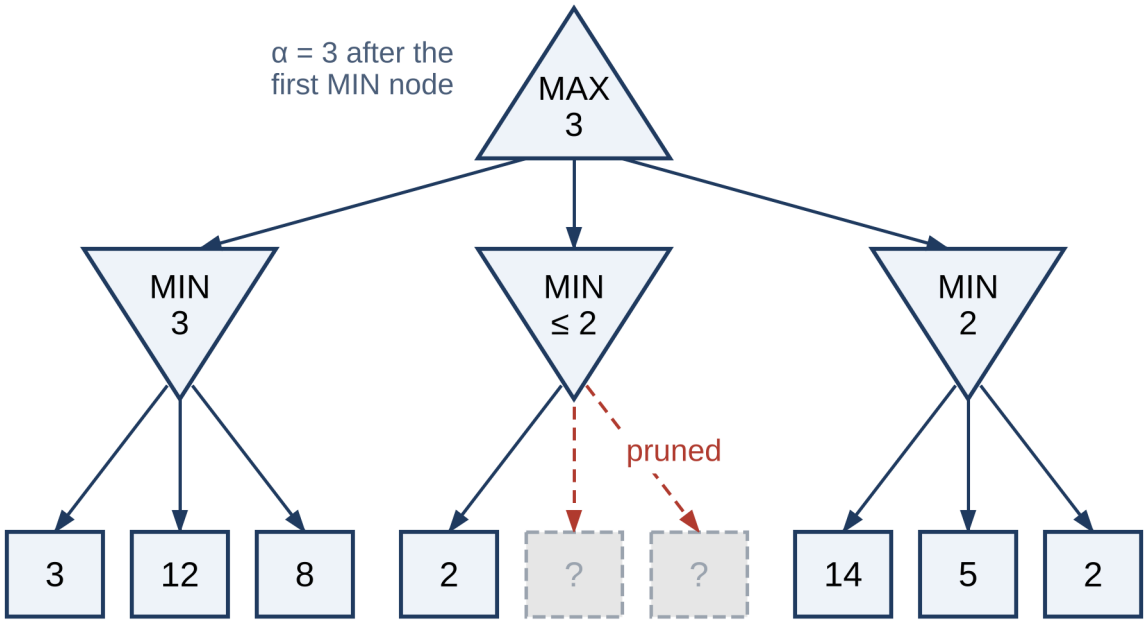


Figure 1.1 Minimax Game Tree with Alpha-Beta Pruning

The difficulty is scale. A chess position offers about 35 legal moves on average, and Shannon estimated the size of the game tree at roughly 10^{120} positions [1]. A plain Minimax search only six plies deep already visits about $35^6 \approx 1.8$ billion leaf positions. Practical engines therefore rely on techniques that shrink the tree without changing the answer — pruning, move ordering and the caching of previously searched positions — together with a time-bounded search strategy and an evaluation function good enough to guide a depth-limited search.

The same worst-case reasoning underpins adversarial thinking in cyber security, where a defender must plan against an intelligent opponent who chooses the most damaging response; attack–defence trees and security games are close relatives of the game tree. Building an adversarial search agent from first principles, and deploying it as a public web application whose own attack surface is analysed, links the Foundations of Artificial Intelligence course with the security focus of the department.

1.2 PROBLEM STATEMENT

Building a chess-playing agent that is both correct and practical raises several connected problems in the search itself. The game tree grows exponentially with depth, so exhaustive search is impossible and plain Minimax reaches only a shallow depth within an interactive time limit. A depth-limited search that stops in the middle of a capture sequence suffers from the horizon effect and can badly misjudge a position, because the recapture lies just beyond the search horizon. The savings of Alpha-Beta pruning also depend heavily on move ordering: without good ordering and a cache of searched positions, much of the possible reduction is lost and positions reached through different move orders are searched again and again.

Two further problems lie outside the search. Many course-level engines rely on an external library for move generation and show only the chosen move, so neither the correctness of the rules engine nor the effect of each optimisation is visible; and an engine delivered as a public web application inherits the security concerns of any web software — the integrity of its build pipeline, the confidentiality of its deployment credentials and the behaviour of the client in the user's browser. The problem addressed by this project is therefore: given any legal chess position and a fixed time budget, select the move with the best depth-limited minimax value using a self-built, verifiably correct move generator and a hand-crafted evaluation function; make the cost of every search optimisation measurable and visible; and deliver the result as a secure, responsive, client-side web application.

1.3 OBJECTIVES

The objectives of the project are:

- To formulate chess as a state-space search problem — states, actions, transition model, terminal test and utility — consistent with intelligent-agent theory.

- To implement a self-contained board representation and legal move generator, including castling, en passant and promotion, without any external chess library.
- To implement Minimax in its Negamax form with Alpha-Beta pruning, driven by Iterative Deepening within a fixed time budget and extended with Quiescence Search.
- To reduce the effective branching factor through hash-move and MVV-LVA move ordering, a fixed-size Zobrist-hashed transposition table and null-move pruning.
- To design a hand-crafted evaluation function that combines material, piece-square tables, pawn structure, king safety and mobility.
- To verify move generation with perft (performance test) node counts on the standard test positions through an automated test suite, and to confirm that pruning, ordering and caching return the same move and score as plain Minimax.
- To measure playing strength with self-play matches and a benchmark against a reference engine of known strength.
- To expose the engine's reasoning through an interactive browser interface with time controls, typed moves and a teaching mode that compares node counts across search configurations on any position.
- To deploy the system as a static web application and analyse its threat model and security controls.

1.4 SCOPE

The project covers the design, implementation, verification and deployment of a classical chess engine and its browser interface. The boundaries of the work are defined below.

1.4.1 IN SCOPE

- The complete rules of chess: all piece movements, castling rights, en passant, promotion to any piece, and detection of checkmate, stalemate, the fifty-move rule, insufficient material and threefold repetition.
- Negamax search with Alpha-Beta pruning, Iterative Deepening (depth 1 to 12 within a time budget), Quiescence Search, MVV-LVA ordering, null-move pruning, configurable contempt and a fixed-size transposition table.
- A five-term evaluation function with a per-term breakdown.

- A React interface running the engine in a Web Worker: a new-game dialog with time controls, side and contempt; chess clocks; premoves (moves queued while the engine is thinking); moves typed in Standard Algebraic Notation (SAN) or UCI; and a teaching mode that accepts any position as FEN.
- Verification and measurement: an automated test suite that gates deployment, experiments over 17 positions, self-play matches, a tactical suite and a playing-strength benchmark against Stockfish.
- Static deployment on Firebase Hosting through a GitHub Actions pipeline, together with a security analysis of the deployed system.

1.4.2 OUT OF SCOPE

- Machine-learning components of any kind: neural-network evaluation, reinforcement learning or training data.
- Maximising playing strength through opening books or endgame tablebases.
- Online or human-versus-human play, user accounts and server-side storage.
- Acting as a UCI engine for external chess graphical interfaces (a UCI client is used only to drive the reference engine in benchmarks).

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

Although the engine is an AI artefact, the delivered product is a public web application and is analysed as such. The system has no backend, no user accounts and no stored personal data, which removes whole classes of risk such as database injection or the theft of user credentials. The remaining concerns are the integrity of the code that reaches the user, the confidentiality of the credential used to publish it, the availability and responsiveness of the client, and the privacy of the player. Table 1.2 lists the security requirements derived from these concerns and their status in the current prototype.

Table 1.2 Security Requirements

ID	Requirement	How it is addressed	Status
SR1	Integrity of the deployed build	Deployment only through the GitHub Actions pipeline; npm ci from the committed lockfile; 53 automated tests must pass before the build	Partially met — the deploy step also runs on pull-request events
SR2	Confidentiality of the deployment credential	Firebase token stored as an encrypted GitHub Actions secret, never in the repository	Met — a scoped service account is recommended
SR3	Transport security	The site is served only over HTTPS by Firebase Hosting	Met
SR4	Resistance to client-side injection	React escapes all rendered text; no innerHTML, eval or user-supplied HTML in the source	Met
SR5	Input integrity	Clicked and typed moves (SAN or UCI) are parsed and checked against the legal-move list; FEN entered in teaching mode is validated, and a bad one is explained, before use	Met
SR6	Availability and responsiveness	Search runs in a Web Worker within a time budget; a fixed-size 10 MB transposition table; teaching-mode searches stop at 60 s	Met
SR7	Privacy of the player	No accounts, cookies, analytics or server; game state never leaves the browser (web fonts are the only third-party request)	Met
SR8	Freshness of cached code	HTML served no-cache, must-revalidate; fingerprinted assets served immutable	Met
SR9	Browser hardening headers	Content-Security-Policy and related response headers	Not yet met — planned

The design follows the principles of least privilege and minimal attack surface: by keeping all computation on the client and publishing only static files, the system exposes no server-side logic to attack. Its main residual risks lie in the software supply chain and the continuous-delivery pipeline, which are analysed in Section 3.4.

1.6 SDG ALIGNMENT

The project contributes to the United Nations Sustainable Development Goals [20], mainly through education and technological capability, as shown in Table 1.3.

Table 1.3 SDG Alignment

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 4 – Quality Education	4.4 – Increase the number of youth and adults with relevant skills, including technical and ICT skills	Teaching mode makes pruning, move ordering and caching visible and measurable, turning abstract search theory into hands-on learning.
2	SDG 4 – Quality Education	4.3 – Equal access to affordable and quality technical and tertiary education	A free web application that runs on any laptop or phone with no installation, account or special hardware.
3	SDG 9 – Industry, Innovation and Infrastructure	9.5 – Enhance scientific research and upgrade technological capabilities	A transparent, openly licensed research artefact with reproducible measurements that others can inspect and extend.
4	SDG 7 – Affordable and Clean Energy	7.3 – Improve the rate of energy efficiency	Classical search needs no training phase and no GPU; the engine runs on a single CPU core inside the browser.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the foundational research on adversarial search in chess, surveys existing engines together with the security approaches used to build and deliver them, identifies the gaps addressed by this project, and introduces the proposed system.

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

2.1.1 FOUNDATIONAL RESEARCH

Shannon (1950) [1] proposed representing chess as a tree searched with Minimax and scored by an evaluation function that combines material with positional factors. He distinguished full-width "Type A" search from selective "Type B" search and showed that full-width search to useful depths is computationally infeasible, which motivated every later pruning technique.

Zobrist (1970) [2] introduced a hashing scheme in which every piece–square combination, the side to move, the castling rights and the en passant file are assigned random bit strings that are combined with exclusive-OR. The resulting keys make it practical to recognise positions reached through different move orders, the basis of transposition tables.

Knuth and Moore (1975) [3] proved that Alpha-Beta pruning returns the exact Minimax value and showed that, with perfect move ordering, it examines about $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaf nodes instead of b^d, roughly doubling the depth that can be searched in the same time. In the worst case, with poor ordering, it saves nothing. Marsland's review [13] surveys the many enhancements that followed.

Korf (1985) [4] showed that depth-first iterative deepening — repeated depth-limited searches of increasing depth — is asymptotically optimal in time while using only linear memory, because the final iteration dominates the total cost. In game playing this gives an "anytime" search that always has a move ready.

Schaeffer (1989) [15] studied the history heuristic and other Alpha-Beta enhancements, and found that move-ordering improvements such as the transposition-table move account for most of the practical gain. **Beal (1990)** [14] generalised quiescence search, which extends the search at the leaves through tactical moves until a quiet position is reached,

reducing the horizon effect. **Donninger (1993)** [16] described null-move pruning, in which the opponent is given a free move at reduced depth and the branch is pruned if the position still fails high; it is effective but unsound in zugzwang positions.

Campbell, Hoane and Hsu (2002) [5] described Deep Blue, which combined a 30-node IBM RS/6000 SP computer with 480 custom chess chips, parallel Alpha-Beta search and an evaluation function of about 8,000 hand-tuned features. **Coulom (2006)** [6] introduced selective backup operators for Monte-Carlo Tree Search, an alternative to Minimax based on sampled play-outs. **Silver et al. (2018)** [8] presented AlphaZero, which learned chess, shogi and Go from self-play with a deep neural network guiding MCTS, and **Nasu (2018)** [9] introduced the NNUE evaluation network, originally for shogi and since adopted by Stockfish [10].

2.1.2 EXISTING SYSTEMS

Deep Blue was a dedicated supercomputer and was never publicly available. **Stockfish** is open-source and distributed as source code and native binaries; it communicates with graphical interfaces through the Universal Chess Interface (UCI) protocol and reports only a score and a principal variation. **AlphaZero** was never released, but the open-source **Leela Chess Zero** follows the same approach and downloads separately trained neural-network weight files. Several online platforms run **browser builds of Stockfish compiled to WebAssembly** for in-browser analysis. At the other end of the spectrum, **typical course-level engines** use a library such as python-chess for move generation and a plain fixed-depth Minimax with a material-only evaluation, as shown in Figure 2.1.

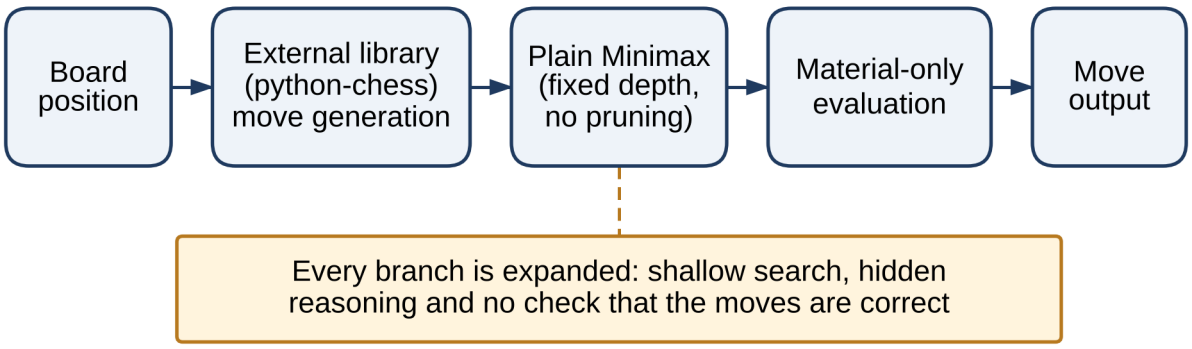


Figure 2.1 Typical Flow of a Basic Course-Level Engine

2.1.3 SECURITY APPROACHES IN EXISTING SYSTEMS

Existing systems address security in ways that match how they are delivered:

- **Native engines** such as Stockfish run as a local process that exchanges only UCI text with a GUI over standard input and output, which keeps the engine isolated from the network. Users must, however, trust the source of the binary they download, and the open-source code allows independent review.
- **Neural engines** such as Leela Chess Zero add a model-provenance concern: the weight files that determine the engine's play are downloaded separately and must come from a trusted source.
- **Browser engines** built with WebAssembly run inside the browser sandbox, benefiting from the same-origin policy and process isolation, and commonly run in a Web Worker so that analysis cannot freeze the page.
- **Online chess platforms** treat engine assistance itself as the main threat to game integrity and operate fair-play systems that compare players' moves with engine recommendations to detect cheating.

Table 2.1 compares these systems with the proposed system across search, evaluation, deployment, transparency and security posture.

Table 2.1 Comparison of Existing Systems

System	Search	Evaluation	Deployment	Transparency	Security posture
Deep Blue	Parallel α-β on custom chips	~8,000 hand-tuned features	Dedicated supercomputer	Closed	Isolated; not publicly accessible
Stockfish	α-β with many enhancements	NNUE neural network	Native binary, UCI	Score and principal variation only	Local process; trust in binary source
AlphaZero / Leela Chess Zero	Neural-guided MCTS	Deep neural network	GPU/TPU; weight files	Learned, not interpretable	Weight-file provenance
Browser (WASM) engines	α-β (Stockfish port)	Neural or classical	Browser sandbox	Score and line only	Sandbox and same-origin policy
Typical student engine	Plain fixed-depth Minimax	Material only	Script or notebook	Code visible, no metrics	Rarely considered

System	Search	Evaluation	Deployment	Transparency	Security posture
Proposed system	Negamax α-β, ID, QS, null move, MVV-LVA, TT	Five-term hand-crafted	Static web app, Web Worker	Live telemetry and teaching mode	Threat-modelled; tests gate deploys

2.1.4 LIMITATIONS OF EXISTING SYSTEMS

- Strong engines are either closed (Deep Blue, AlphaZero) or depend on learned evaluation that cannot be explained move by move.
- Engines expose only their final score and line, not the work that each technique saved or cost.
- Neural engines need GPUs or TPUs for training and, for best play, for inference.
- Course-level engines seldom implement or verify their own move generation, and rarely use pruning, ordering or caching.
- Security analysis is almost absent from academic engine projects, even when they are published on the open web.

2.2 RESEARCH GAPS

The review above reveals five gaps that the project sets out to close:

- **G1 – Transparency.** Strong engines report a score and a principal variation, but not what pruning, ordering or caching contributed. The cost of each technique is invisible to learners.
- **G2 – Dependence on learned components and special hardware.** State-of-the-art strength now comes from neural networks trained on GPUs or TPUs, which are not interpretable and are unnecessary for teaching classical search.
- **G3 – Unverified correctness in course projects.** Engines built on an external move-generation library are neither implemented nor verified by the student, and verification by perft (performance test), which counts every possible move sequence to a fixed depth, is rarely applied.
- **G4 – Lack of controlled comparison.** Few tools search the same position at the same depth under different configurations, confirm that all of them return the same result, and report the cost of each.

- **G5 – Security of AI deliverables.** Academic AI projects seldom analyse the attack surface of how they are built and delivered, even when they are publicly hosted.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system is a fully self-built, classical chess engine delivered as a client-side web application. The board is represented as a 64-square array together with the side to move, castling rights, en passant square and move clocks, and can be read from Forsyth–Edwards Notation (FEN). A move generator produces pseudo-legal moves for every piece type and discards any move that would leave the mover's own king in check.

Moves are chosen by an Iterative Deepening driver that runs Negamax search with Alpha-Beta pruning at depth 1, 2, 3 and upwards, to a ceiling of 12, until its time budget expires, so a legal best move is always available. Before expanding a node, the search checks for a repeated position or the fifty-move rule — a draw is scored with a configurable contempt — and probes a fixed-size, Zobrist-hashed transposition table of about 10 MB that is kept for the whole game. Where at least four plies remain and the side to move is not in check, null-move pruning lets the opponent move twice and prunes the node if the reduced search still fails high. Moves are ordered with the hash move first, then captures by Most Valuable Victim – Least Valuable Attacker (MVV-LVA), then promotions. At the depth limit, Quiescence Search follows captures until the position is quiet, and the hand-crafted evaluation function scores it. Figure 2.2 shows this flow.

The engine runs in a Web Worker so that the interface never freezes. Each match starts from a new-game dialog in which the player chooses a time control (bullet, blitz, rapid or no clock), a side and the engine's contempt. The play screen offers legal-move hints, moves typed in SAN or UCI, premoves while the engine thinks, chess clocks, an evaluation bar, a SAN move history and an engine console that streams every deepening iteration. A teaching mode searches the current position — or any position pasted as FEN, or one of five sharp presets — at a fixed depth of five under three configurations and compares their node counts and times. An automated suite of 53 tests, including perft on the seven standard test positions, runs before every deployment.

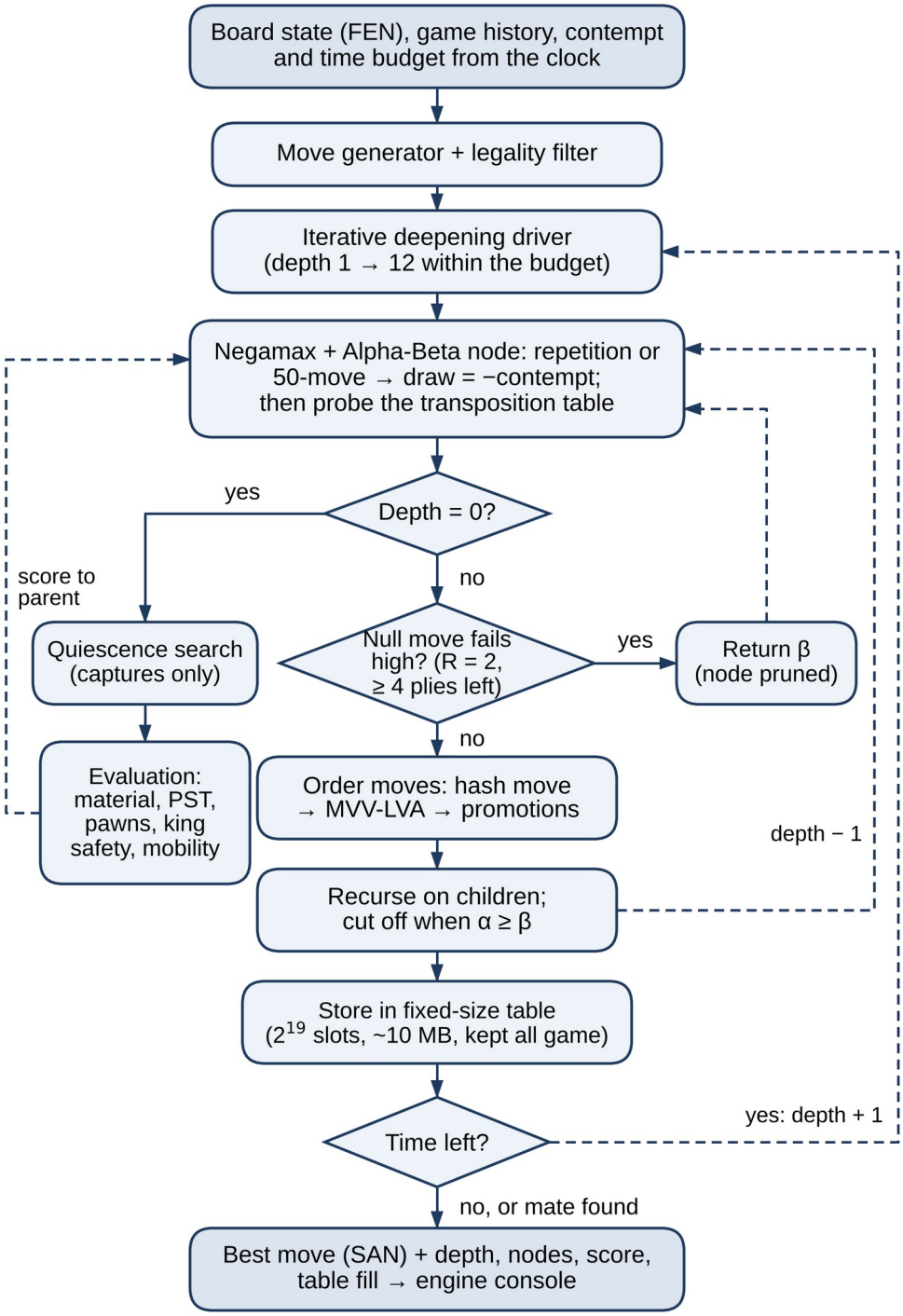


Figure 2.2 Search Flow of the Proposed System

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- Return a legal best move for any position within the time budget set by the clock, without ever blocking the user interface.
- Match plain Minimax exactly: Alpha-Beta, move ordering and the transposition table must return the same move and score at the same depth.

- Verify move generation by perft against the published counts for the standard test positions, automatically on every build.
- Measure playing strength in games, not only in node counts.
- Make the cost of each technique visible through live telemetry and a teaching-mode comparison.
- Play complete legal games to a named ending with the user taking either colour.
- Run in any modern browser, including on phones, with no server, account or installation, and with a documented security posture.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

- **Interpretable:** every decision can be traced to search statistics and to a five-term evaluation breakdown.
- **Lightweight:** no machine learning, training data or GPU is needed; the engine runs on one CPU core in the browser.
- **Self-verifying:** 53 automated tests — perft on seven standard positions, FEN, SAN, repetition, evaluation symmetry, the transposition table and both searches — must pass before any deployment.
- **Measurably efficient:** across 17 positions, Alpha-Beta with ordering and the transposition table cuts the depth-4 tree about 110-fold (geometric mean) with exactly the same results; null-move pruning adds about 130 Elo in self-play.
- **Benchmarked:** the engine plays at roughly 1600–1700 on Stockfish's calibrated strength scale at one second per move.
- **Responsive and portable:** off-thread search and static deployment make the system usable on laptops and phones alike.
- **Minimal attack surface:** there is no backend, no account and no stored personal data, and the delivery pipeline has been analysed for threats.
- **Modular:** search enhancements can be switched on and off independently for controlled experiments.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

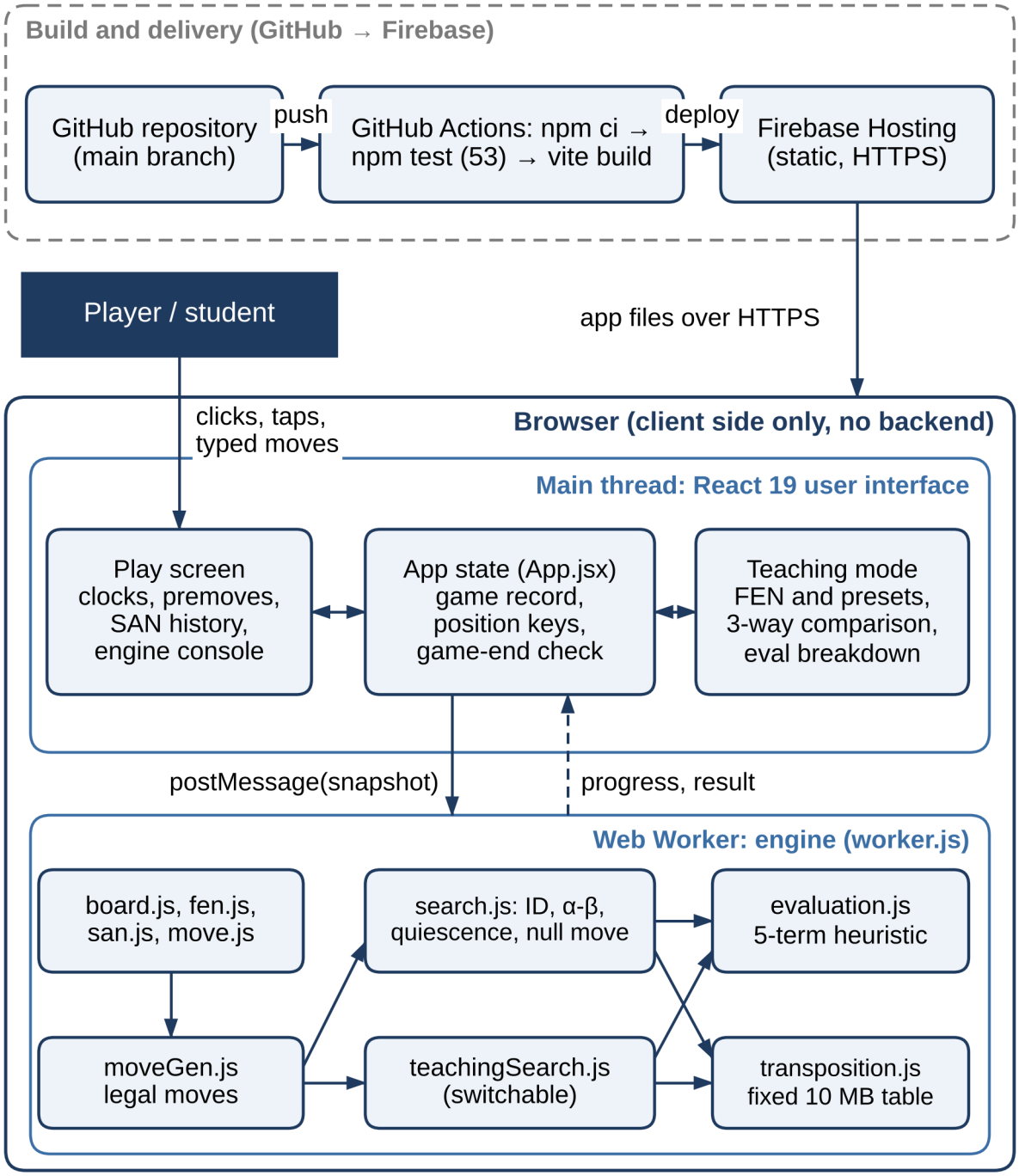


Figure 3.1 System Architecture

Figure 3.1 shows the architecture of the system. It runs entirely on the client and has three parts: the build-and-delivery pipeline, the React user interface on the browser's main thread, and the chess engine in a dedicated Web Worker.

- **Build and delivery.** A push to the main branch of the GitHub repository triggers a GitHub Actions workflow that installs dependencies with npm ci, runs the 53-test suite, builds the production bundle with Vite and publishes it to Firebase Hosting, which serves the static files over HTTPS. A failing test stops the deployment.
- **User interface (main thread).** App.jsx holds the game record, the clocks, the list of position keys used for repetition detection and the game-end logic; the new-game dialog, the play screen and the teaching mode render from this state.
- **Engine (Web Worker).** worker.js accepts three messages — search, teach and legalMoves — rebuilds the board from a serialised snapshot and streams progress and results back. Because the board is passed as a copy, the worker never shares mutable state with the interface. The worker keeps one fixed-size transposition table for the whole game and clears it when the engine's side or contempt changes.

Separating the engine from the interface keeps the page responsive: the main thread only renders, while the search, which may examine hundreds of thousands of positions per move, runs in parallel on another thread. The hardware and software requirements are listed in Tables 3.1 and 3.2.

Table 3.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent; any recent smartphone	Intel Core i5 or above
Memory (RAM)	4 GB (the engine's transposition table is a fixed 10 MB)	8 GB, for teaching-mode plain-Minimax runs
Storage	Browser cache only (production bundle ≈ 0.4 MB)	—
GPU	Not required	Not required
Network	Internet connection for the first load	Broadband connection
Development machine	Laptop capable of running Node.js 20	—

Table 3.2 Software Requirements

Category	Technology	Purpose
Language	JavaScript (ES modules)	Engine and interface
UI framework	React 19	Components and state for the play and teaching screens
Build tool	Vite 8	Development server and production bundling
Concurrency	Web Workers API	Search off the main thread
Animation	framer-motion	Interface transitions
Development runtime	Node.js 20, npm	Dependency installation and builds
Testing	Node.js built-in test runner (node:test)	53 automated tests; CI gate before every build
Benchmarking (not shipped)	Stockfish 19 (WebAssembly build, npm package)	Reference opponent of calibrated strength
Version control and CI/CD	Git, GitHub, GitHub Actions	Source control and automated deployment
Hosting	Firebase Hosting	Static HTTPS hosting
Editor	Visual Studio Code	Development
Client	Any modern browser (Chrome, Edge, Firefox, Safari)	Running the application

3.2 MODULE DESIGN

The system is organised into six modules. Table 3.3 maps each module to its source files; each module is then described in turn together with its Level-2 data flow diagram (DFD). The Level-0 and Level-1 diagrams that place the modules in context are given in Section 3.3.1.

Table 3.3 Module-to-File Mapping

Module	Source files	Responsibility
M1 – Board representation and move generation	board.js, moveGen.js, move.js, fen.js, san.js	Game state, FEN in/out and validation, SAN in/out, make/unmake, legal moves, repetition keys
M2 – Heuristic evaluation	evaluation.js	Five-term static evaluation and per-term breakdown

Module	Source files	Responsibility
M3 – Adversarial search	search.js	Iterative deepening, Negamax α-β, quiescence, null move, mate and draw scoring with contempt
M4 – Search optimisation	transposition.js, search.js, teachingSearch.js	Zobrist hashing, fixed-size transposition table, move ordering, switchable techniques
M5 – Interface, teaching mode and deployment	App.jsx, components/, premove.js, worker.js, deploy.yml, firebase.json	User interface, clocks, premoves, worker messaging, build and hosting configuration
M6 – Verification and benchmarking	test/, bench/	Automated tests; experiments, matches, tactical suite, Elo benchmark

3.2.1 BOARD REPRESENTATION AND MOVE GENERATION

The board is a 64-element array indexed from 0 (a8) to 63 (h1), in which each square holds a piece character — upper case for White, lower case for Black and '.' for empty. The board also records the side to move, the four castling rights, the en passant target square, the half-move clock and the full-move number, together with a history stack that allows every move to be undone exactly. makeMove() updates all of these, including removing castling rights when a king or rook moves or when a rook is captured on its home corner; unmakeMove() restores them.

A move object records its start and target squares, the piece moved, any piece captured, castling and en passant flags and the promotion piece. san.js writes moves in Standard Algebraic Notation — with disambiguation, check and mate marks — and reads typed moves in SAN or UCI, forgiving harmless variations such as a missing capture sign and explaining why a move cannot be played (illegal, ambiguous, or a promotion with no piece named). fen.js validates a FEN string before it is used, rejecting wrong rank lengths, missing kings, pawns on the back rank, castling rights without king and rook at home, impossible en passant squares and a side not to move that is in check.

The generator produces pseudo-legal moves for each piece type — sliding moves for bishops, rooks and queens with checks that stop them wrapping around the board edge, the eight knight offsets, king steps, and pawn pushes, double pushes, captures, en passant captures and promotions to queen, rook, bishop or knight. Castling is generated only when the king and the squares it crosses are not attacked. Each pseudo-legal move is then made,

tested with isInCheck() and unmade, and only moves that leave the mover's king safe are returned. For repetition detection, positionKey() builds an exact string of placement, side to move, castling rights and en passant square — the last only when a capture en passant is actually possible.

Figure 3.2 shows the Level-2 data flow diagram of this module. A FEN string is validated (1.1) before it reaches the board state (D1); a typed move is read by the SAN and UCI parser (1.2), matched against the legal moves from the legality filter (1.5) and applied by make and unmake (1.3). The generator (1.4) and the filter (1.5) supply legal moves to the search, and every position played is keyed (1.6) into the history of positions played (D2) that is used to detect repetition.

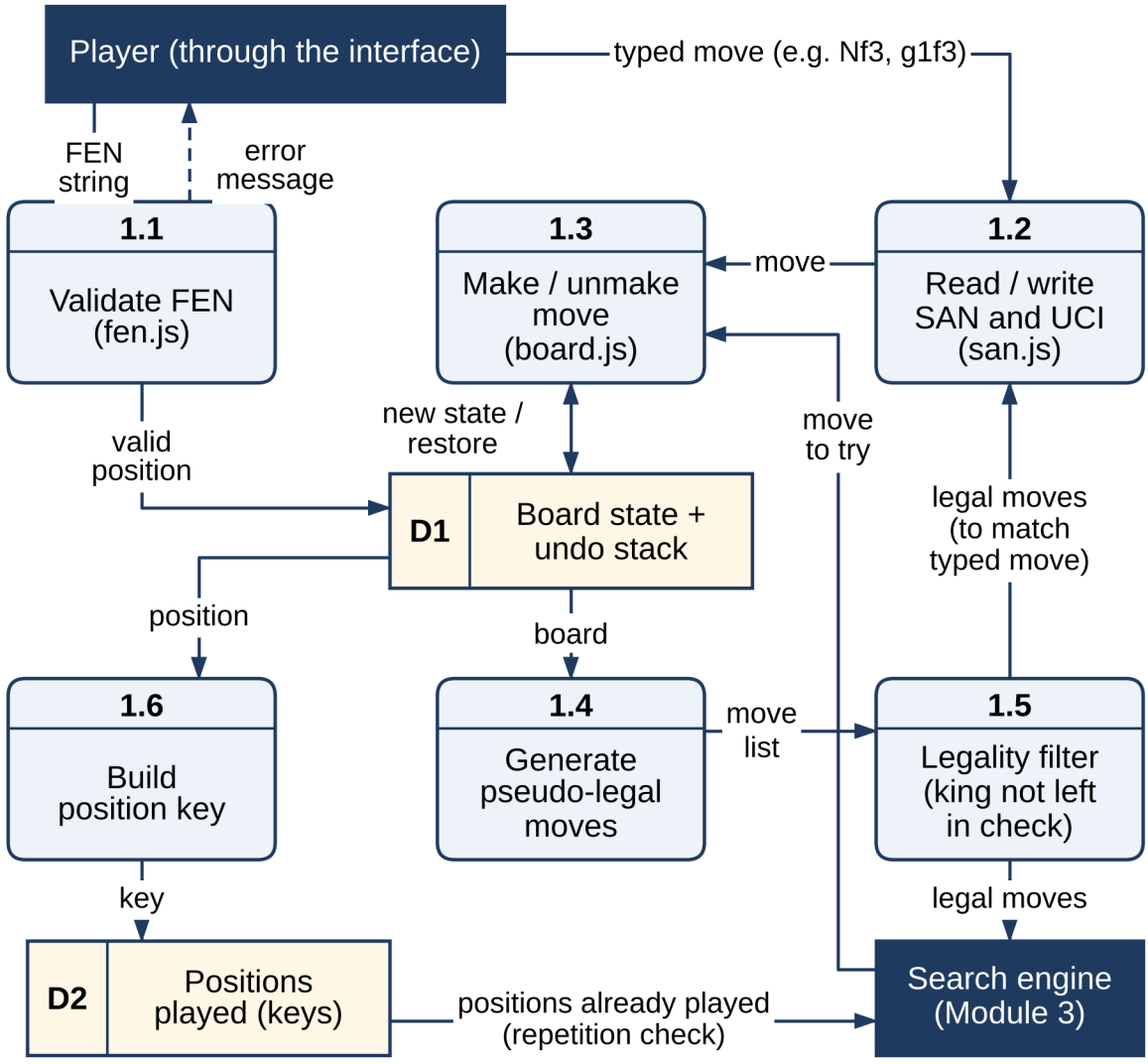


Figure 3.2 Level-2 DFD of Module 1 – Board Representation and Move Generation

3.2.2 HEURISTIC EVALUATION FUNCTION

The evaluation returns a score in centipawns from White's point of view. It adds five terms: material; piece-square values, with Black's tables mirrored; a pawn-structure penalty for doubled and isolated pawns; a king-safety term that penalises open files at the king and rewards a pawn shield when the king has castled to a wing; and a mobility bonus for knights and bishops. The same code path produces both the single score used by the search and the per-term breakdown shown in teaching mode, so the two can never disagree. The exact terms are listed in Table 4.2.

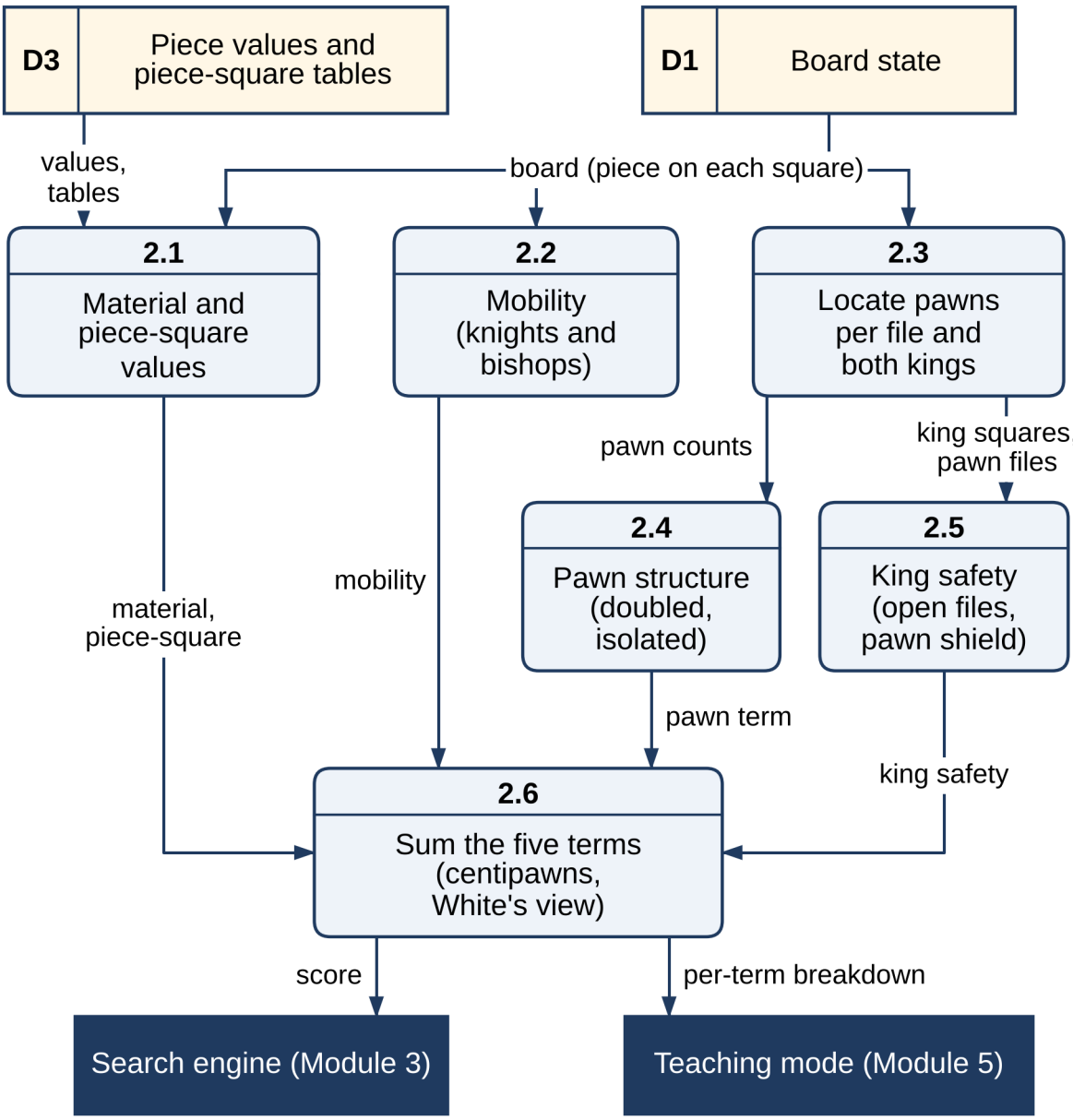


Figure 3.3 Level-2 DFD of Module 2 – Heuristic Evaluation Function

Figure 3.3 shows the data flow of the evaluation. A single pass over the board state (D1) scores material and piece-square values from the tables (D3), counts the mobility of knights and bishops, and records the pawns on each file and the squares of both kings. The pawn-structure and king-safety terms are computed from those counts, and the five terms are summed (2.6) into one score for the search and a per-term breakdown for teaching mode.

3.2.3 ADVERSARIAL SEARCH ENGINE

getBestMove() runs Iterative Deepening from depth 1 to 12. Each iteration calls a root search that orders the legal moves, searches each with Negamax and records the best. The node counter checks the clock every 2,048 nodes and aborts the search when the budget is spent; the move from the last completed iteration is kept. Without a clock the budget is 2,000 ms; in timed games it is about one-thirtieth of the engine's remaining time plus most of the increment, between 50 ms and 2,000 ms. The search stops early when it finds a forced mate (a score whose magnitude exceeds 15,000). Checkmate is scored as −20,000 plus the distance from the root, so shorter mates are preferred. A repetition, stalemate or fifty-move draw inside the tree is worth −contempt to the engine, where contempt ranges from −100 (seek draws) to +100 centipawns (avoid them).

Null-move pruning lets the side to move pass: if the opponent, moving twice and searched two plies shallower (R = 2) with a null window at β, still cannot bring the score below β, the node is cut. It is tried only with at least four plies left, never in check, never twice in a row, not near mate scores, and not when the side to move has only king and pawns, where zugzwang is common. The four-ply guard was chosen by experiment (Section 4.2.2).

Figure 3.4 shows how a search request flows through the module. The interface sends the board snapshot, position history, time budget and contempt; iterative deepening (3.1) drives the root search (3.2), which calls the Negamax node (3.3) for every child. The node consults the positions played (D2) for repetitions, the null-move test (3.4) for early cut-offs, the move generator for legal moves and the transposition table (D4) for stored results, and hands depth-0 positions to quiescence search (3.5), which asks the evaluation for static scores.

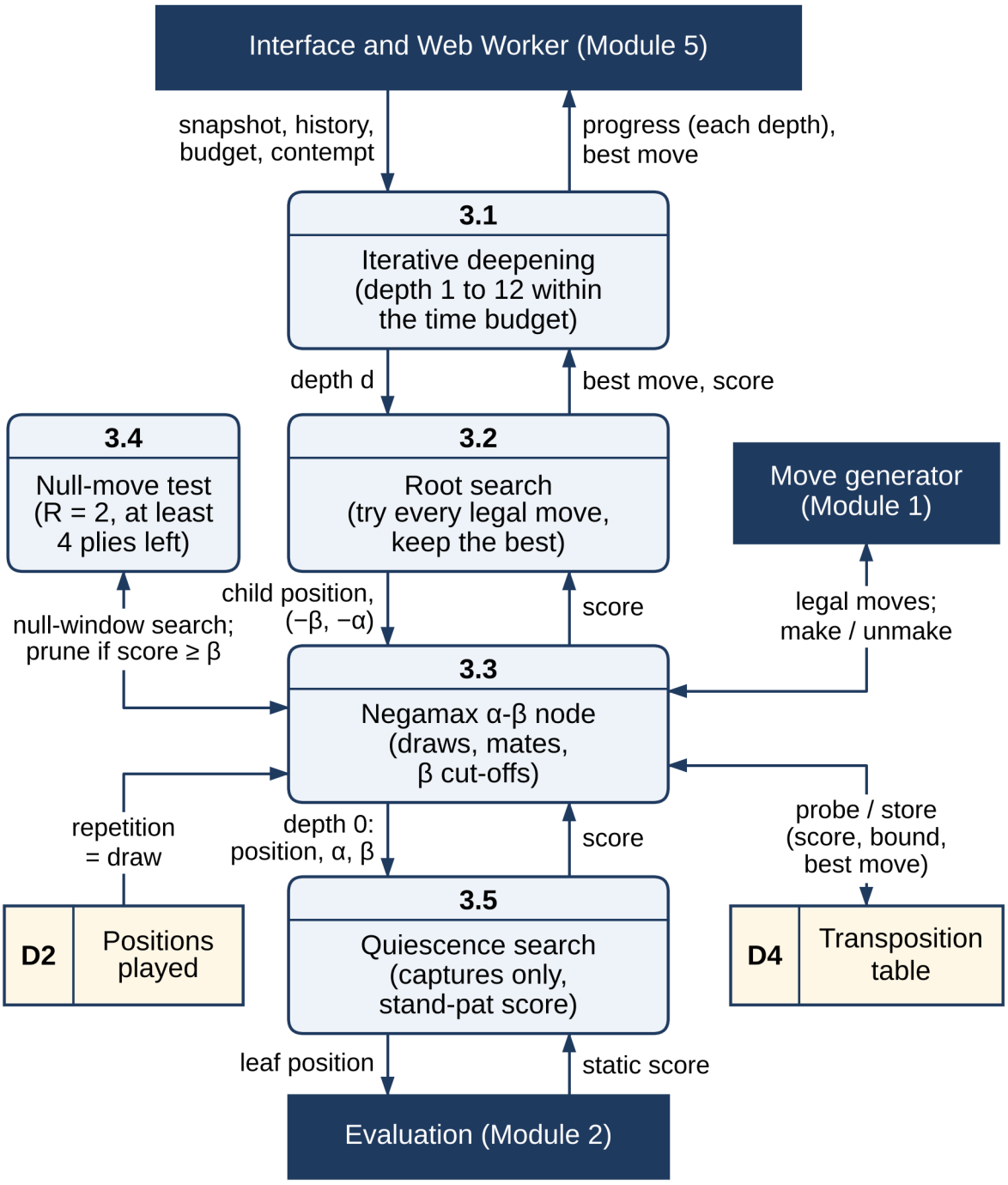


Figure 3.4 Level-2 DFD of Module 3 – Adversarial Search Engine

3.2.4 SEARCH OPTIMISATION

The transposition table is a fixed-size structure of seven parallel typed arrays with 2^{19} slots (about 10 MB), allocated once and never grown. Each slot holds the two 32-bit halves of the Zobrist key, the score, a packed best move, the depth, a bound flag (exact, upper or lower) and a search age. A slot is chosen by the low bits of the key and verified against all 64 bits. When two positions compete for a slot, the same position is always refreshed, an entry from

an earlier search always gives way, and within one search the deeper result stays. Mate scores are stored relative to the node and converted back by ply. A stored score is used only when its depth is sufficient; otherwise only its best move is used, for ordering. Moves are ordered hash move first, then captures by MVV-LVA, then promotions. Quiescence Search evaluates the "stand-pat" score and then searches captures only, in the same order. A separate teaching search (teachingSearch.js) runs at a fixed depth with each technique — Alpha-Beta, move ordering, transposition table, quiescence and null-move pruning — switchable independently.

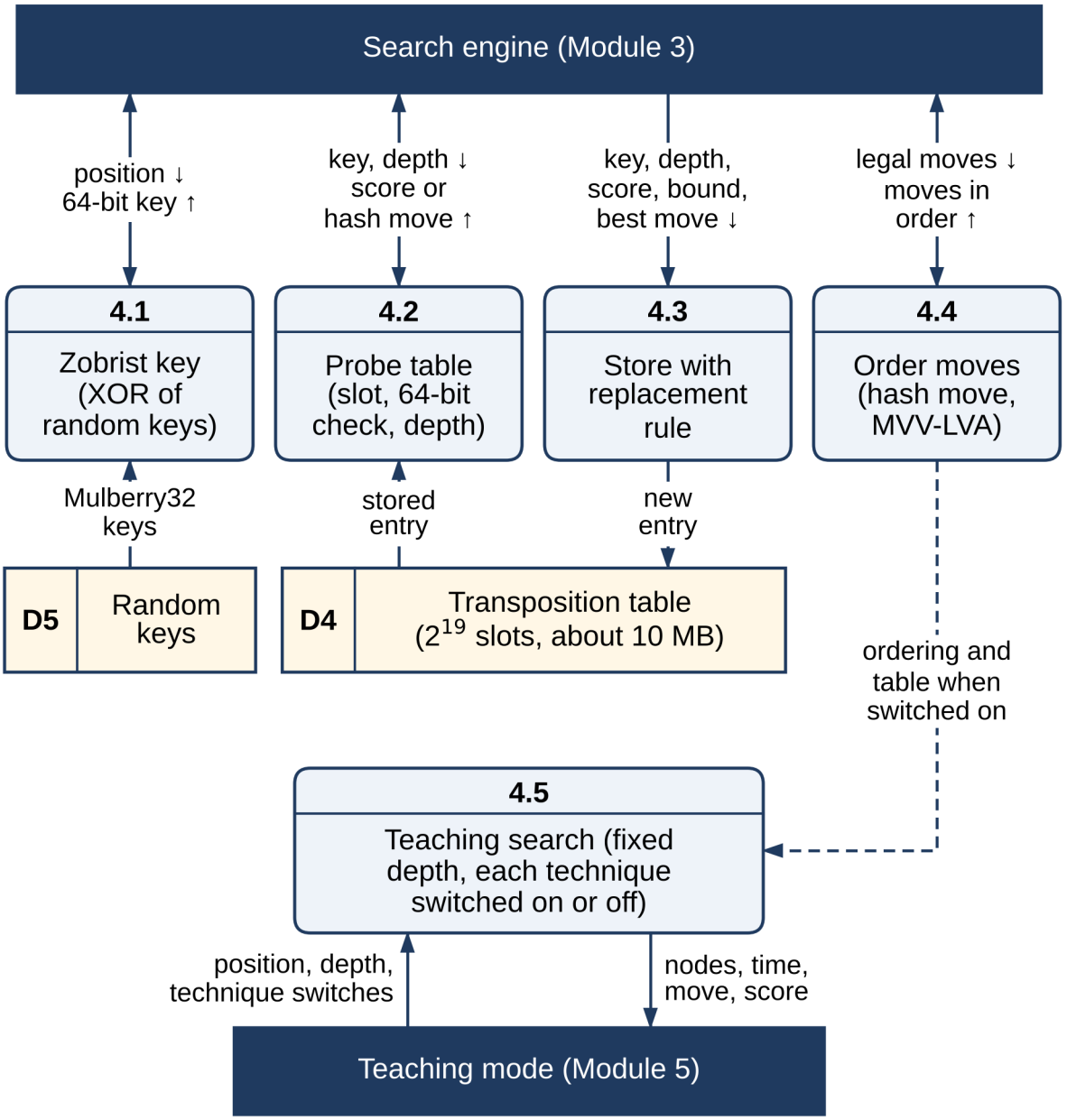


Figure 3.5 Level-2 DFD of Module 4 – Search Optimisation

Figure 3.5 shows the optimisation services that the search calls. Zobrist hashing (4.1) turns a position into a 64-bit key using the random keys (D5); probing (4.2) and storing with

the replacement rule (4.3) read and write the transposition table (D4); and move ordering (4.4) puts the hash move first and then the captures by MVV-LVA. The teaching search (4.5) uses the same ordering and table when they are switched on.

3.2.5 INTERFACE, TEACHING MODE AND DEPLOYMENT

Every match opens with a new-game dialog: a time control (bullet 1+0 or 2+1, blitz 3+0, 3+2 or 5+0, rapid 10+0 or 15+10, or no clock), a side (White, Black or Random) and the engine's contempt. The play screen shows legal targets when a piece is selected, accepts moves typed in SAN or UCI, lets the player queue a premove while the engine thinks, runs chess clocks (a flag loses on time), highlights the last move and a king in check, asks for the promotion piece, and offers undo in untimed games and resignation. It displays an evaluation bar (forced mates shown as M<n>), captured material, a SAN move history and an engine console that lists every deepening iteration and how full the transposition table is.

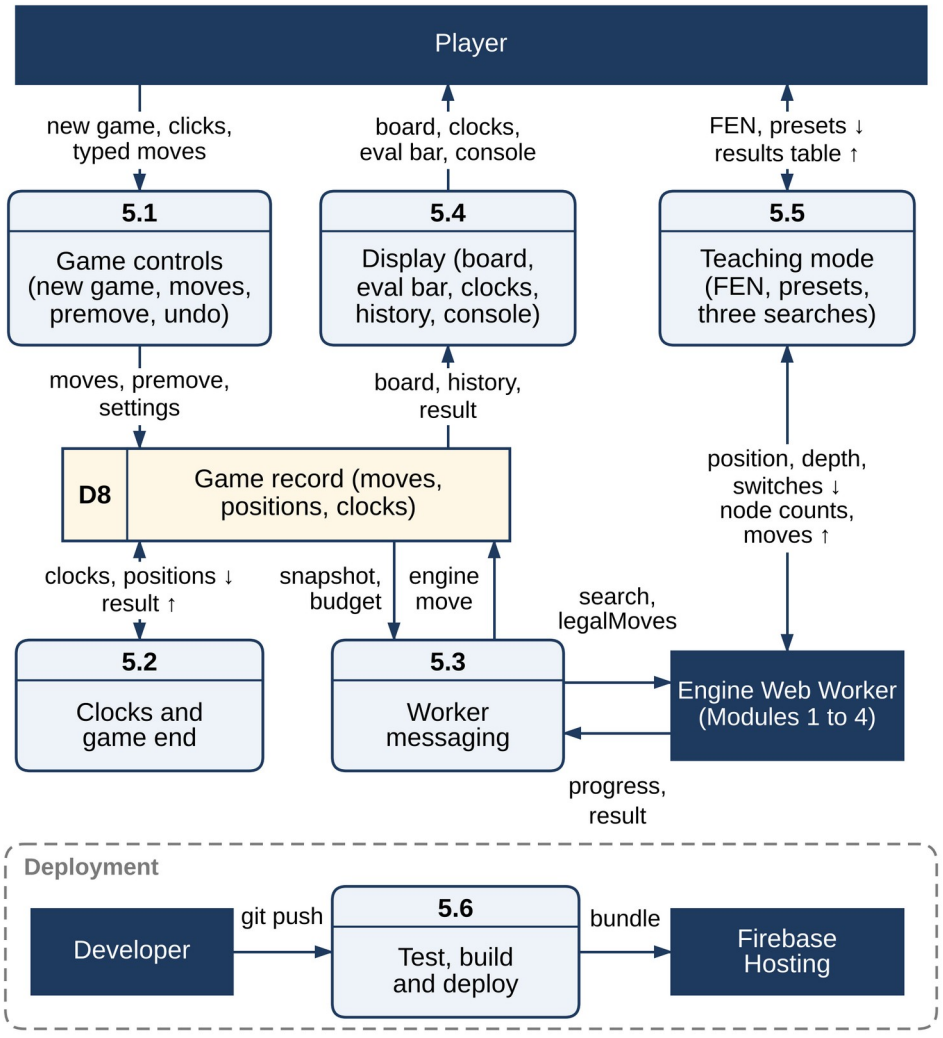


Figure 3.6 Level-2 DFD of Module 5 – Interface, Teaching Mode and Deployment

Teaching mode snapshots the current position — or loads any validated FEN, or one of five presets (the Opera Game mate in two, Win at Chess #1 and #4, the perft (performance test) rook endgame and Kiwipete) — and searches it at depth 5 under three configurations: plain Minimax, Minimax with Alpha-Beta, and a third configuration controlled by the technique toggles. Quiescence and null-move pruning depend on the Alpha-Beta window and are disabled without it. Searches run one at a time in a dedicated worker; a search that passes 60 seconds stops and reports its node count as a lower bound. Deployment is handled by a GitHub Actions workflow and firebase.json, which sets the cache headers.

Figure 3.6 shows the data flow of the interface. Game controls (5.1) record moves, premoves and settings in the game record (D8); the clock and game-end process (5.2) reads it and records the result; worker messaging (5.3) sends the board snapshot and time budget to the engine worker and writes the engine's reply back; and the display (5.4) renders the board, clocks, evaluation bar and console. Teaching mode (5.5) talks to its own worker. The deployment pipeline (5.6) tests, builds and publishes the application to Firebase Hosting.

3.2.6 VERIFICATION AND BENCHMARKING

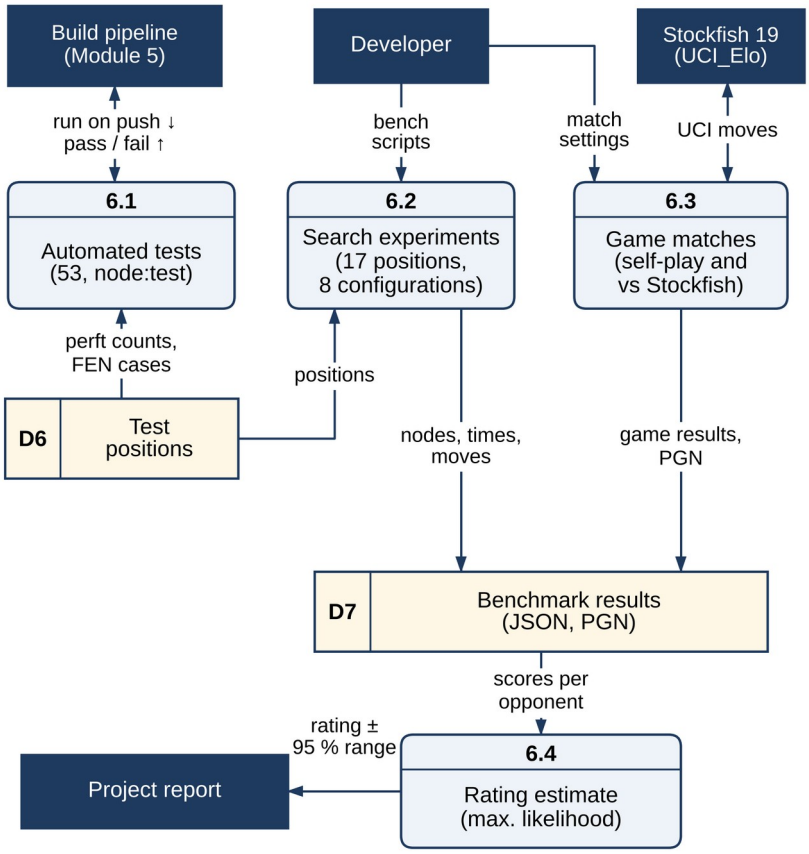


Figure 3.7 Level-2 DFD of Module 6 – Verification and Benchmarking

The test/ directory holds 53 automated tests run with Node's built-in test runner, with no extra dependencies: perft on the seven standard positions, FEN round trips and validation messages, SAN reading and writing, repetition detection, evaluation colour symmetry, the transposition table's size, bounds and replacement rules, and both searches, including mates, contempt and null-move behaviour. The bench/ directory, which is not shipped to users, contains the experiment runner, a match runner for self-play and for games against Stockfish, a generator for a Stockfish-verified tactical suite and a transposition-table sizing study, together with their raw results.

Figure 3.7 shows the data flow of verification and benchmarking. The automated tests (6.1) run on every push and gate the deployment; the experiment runner (6.2) and the match runner (6.3) read the test positions (D6), play against Stockfish where needed and write their results (D7), from which the rating is estimated (6.4) for the report.

3.3 DATA FLOW/UML DIAGRAMS

The behaviour of the system is described with data flow diagrams at Level 0 and Level 1, which place the six modules of Section 3.2 in context (the Level-2 diagram of each module was given with the module), and with three UML diagrams: a use case diagram, an activity diagram and a sequence diagram. In the data flow diagrams, dark rectangles are external entities, rounded boxes are numbered processes, light-yellow rectangles with an identifier on the left are data stores (D1–D8) and labelled arrows are data flows.

3.3.1 DATA FLOW DIAGRAMS

Figure 3.8 is the context (Level-0) diagram. The whole application is a single process that exchanges data with four external entities: the player, who supplies moves, FEN positions and settings and receives the board, the engine's moves and the analysis; the developer, who supplies the source code and receives test and benchmark results; Firebase Hosting, which receives the tested bundle and serves the application files; and Stockfish 19, which is used only as a benchmark opponent.

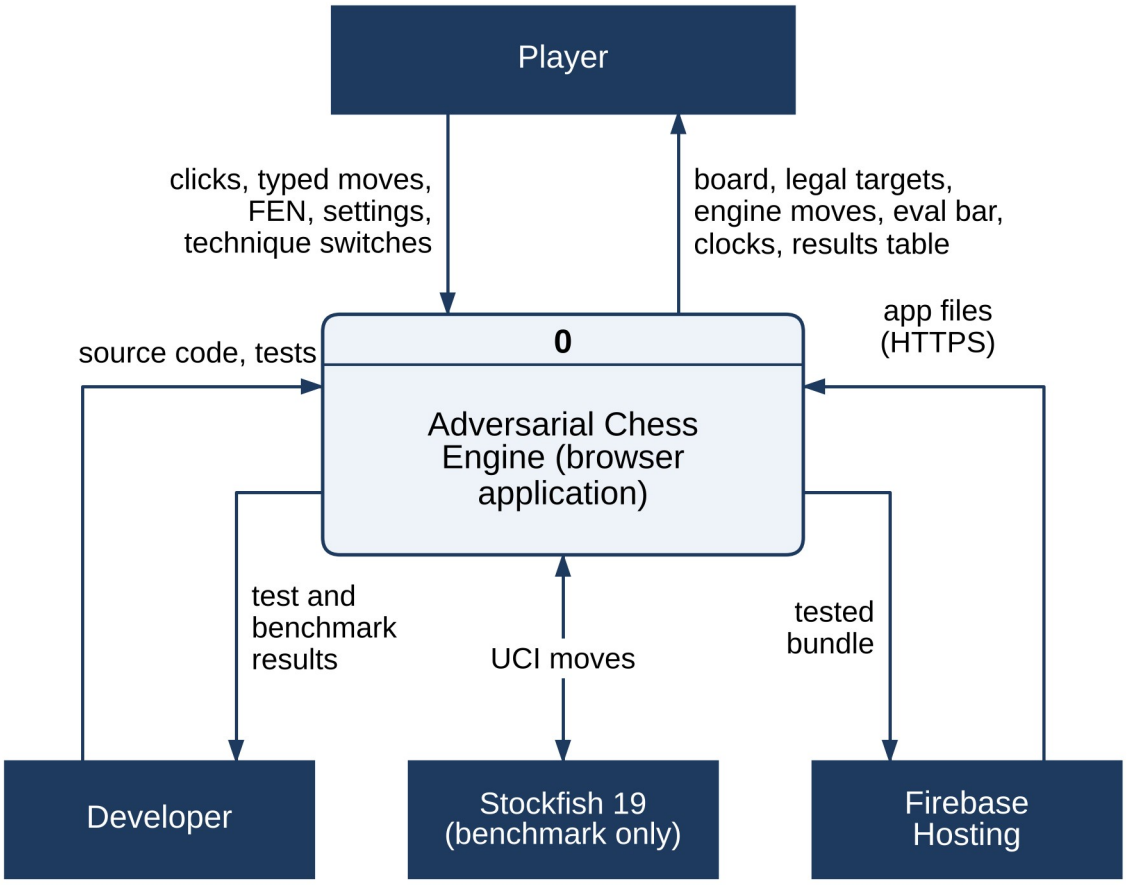


Figure 3.8 Data Flow Diagram – Level 0 (Context Diagram)

Figure 3.9 decomposes the system into the six modules of Section 3.2. The interface (5.0) exchanges the board snapshot and search results with the search (3.0), which calls the move generator (1.0), the evaluation (2.0) and the optimisation services (4.0). The board state (D1), the evaluation tables (D3) and the transposition table (D4) are the shared data stores, and verification (6.0) checks the move generator with perft and plays the benchmark games.

The eight data stores that appear across the diagrams are: D1, the board state with its undo stack; D2, the keys of the positions already played, used to detect repetition; D3, the piece values and piece-square tables of the evaluation; D4, the fixed-size transposition table; D5, the random Zobrist keys; D6, the test positions with their published perft counts; D7, the raw benchmark results in JSON and PGN; and D8, the game record kept by the interface. D1 to D5 are held by the engine in the Web Worker, D8 by the interface on the main thread, and D6 and D7 exist only in the repository and are never shipped to users.

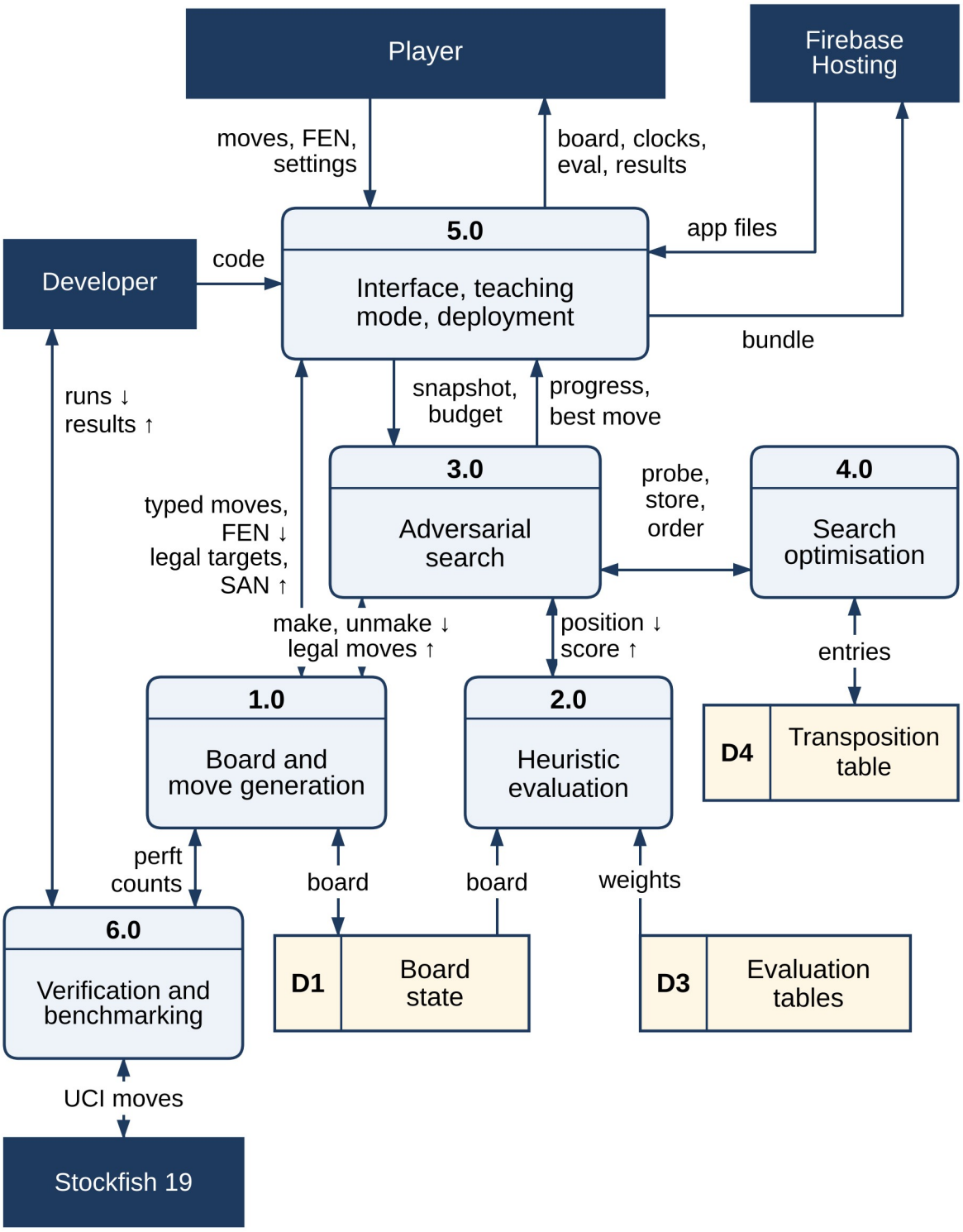


Figure 3.9 Data Flow Diagram – Level 1

3.3.2 USE CASE DIAGRAM

Figure 3.10 shows the two actors. The player starts a game with a time control, side and contempt, requests move hints, makes moves by clicking or typing them, queues premoves, undoes or resigns, reads the evaluation, clocks and telemetry, and works in

teaching mode on the current game or any FEN position. The engine, running in the Web Worker, generates legal moves, parses typed moves and validates FEN, searches for the best move and runs the fixed-depth comparisons. Making a move includes both the engine search and the detection of the end of the game.

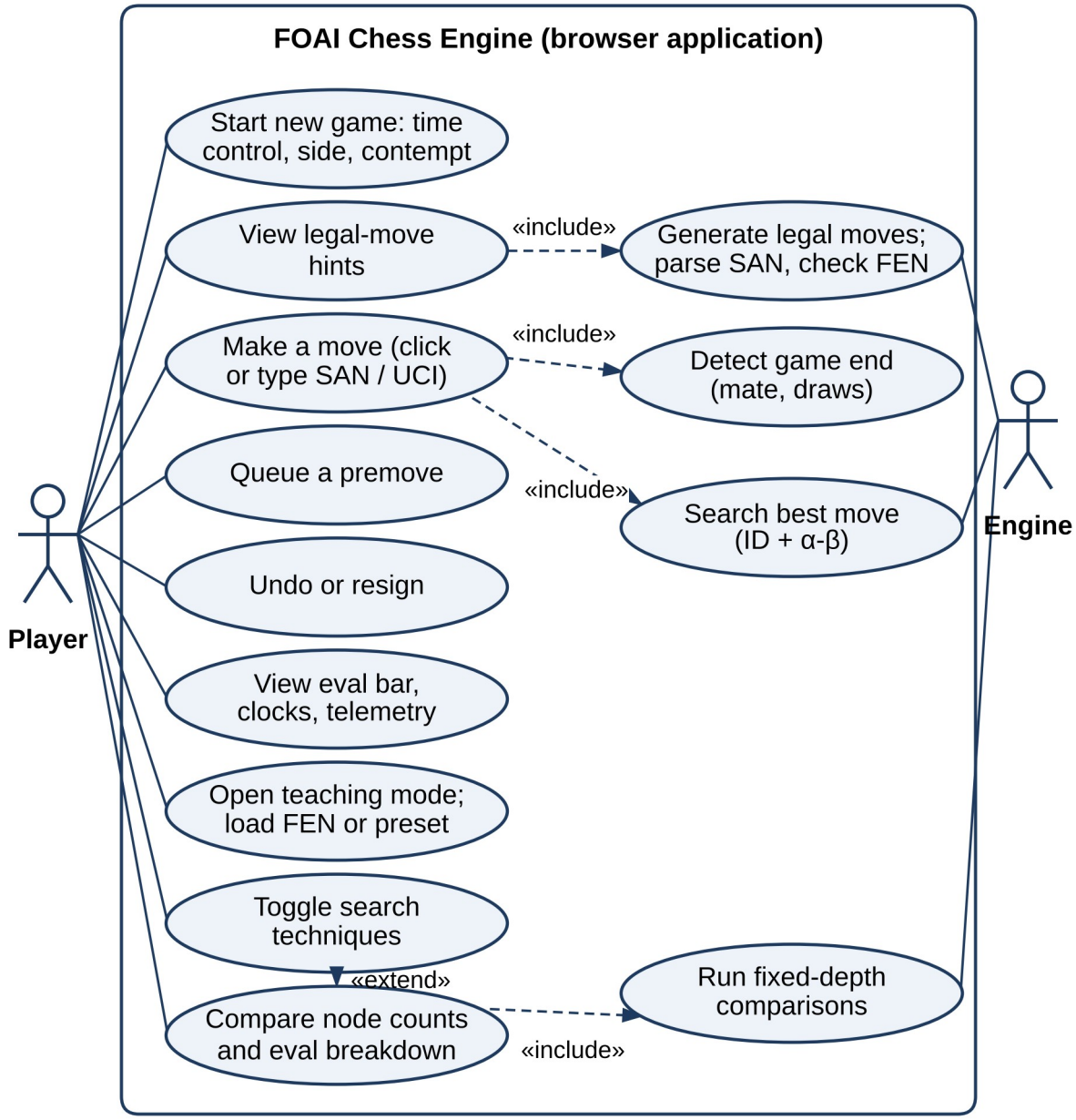


Figure 3.10 Use Case Diagram

3.3.3 ACTIVITY DIAGRAM

Figure 3.11 traces one match, with one swimlane for the main thread and one for the Web Worker. After the new-game dialog, each player move — clicked or typed — is validated against the legal-move list; an illegal or ambiguous move is explained rather than silently ignored. The move is applied, with a promotion choice if needed, and the game-end

check follows. If the game continues, the board snapshot, position history, contempt and time budget are sent to the worker; the engine searches and replies, a premove that the player queued while the engine was thinking is played if it is still legal, and the interface checks again for the end of the game.

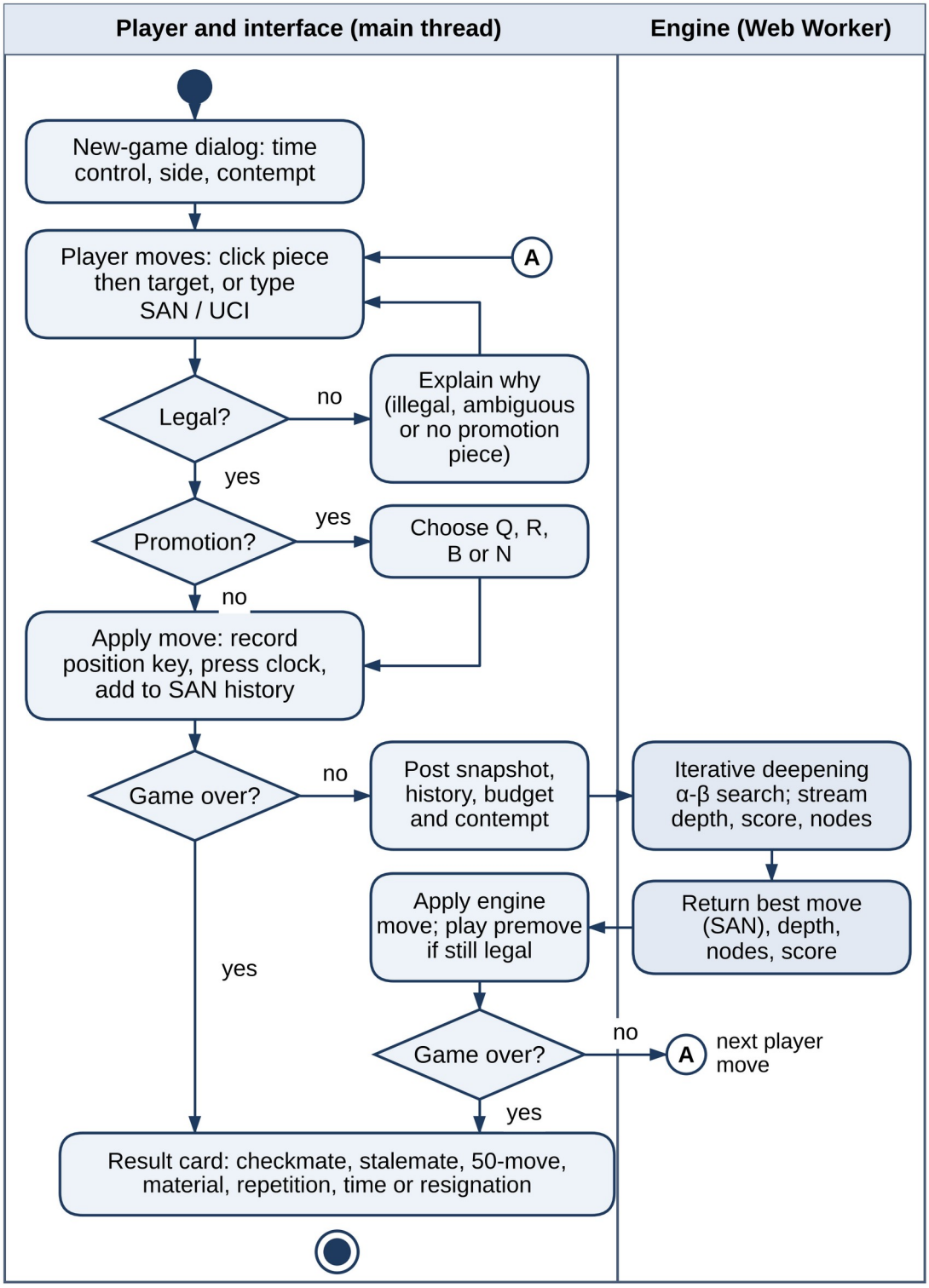


Figure 3.11 Activity Diagram

3.3.4 SEQUENCE DIAGRAM

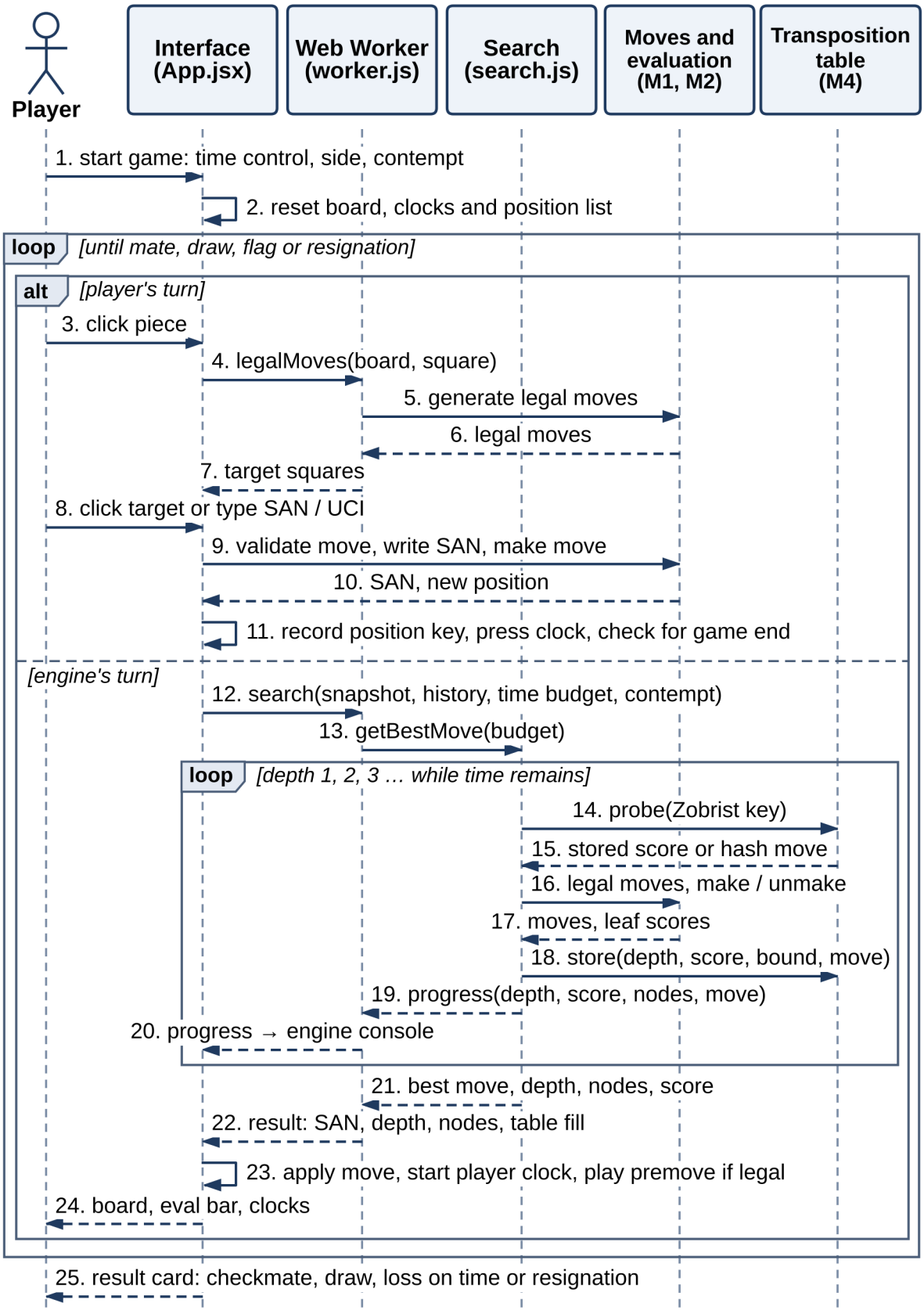


Figure 3.12 Sequence Diagram

Figure 3.12 shows the messages exchanged during a game, numbered in the order in which they occur. After the new-game settings are applied (1–2), each turn belongs either to the player or to the engine. On the player's turn the interface asks the worker for the legal targets of the selected piece (3–7), then validates the move, writes it in SAN and applies it (8–11). On the engine's turn the interface posts the board snapshot, position history, time budget and contempt (12–13); inside the iterative-deepening loop the search probes and updates the game's transposition table, generates and scores moves, and streams progress to the engine console (14–20), before the final result is returned in SAN and applied together with any queued premove (21–24). When the game ends, the result card is shown (25).

3.4 THREAT MODEL AND ATTACK SURFACE

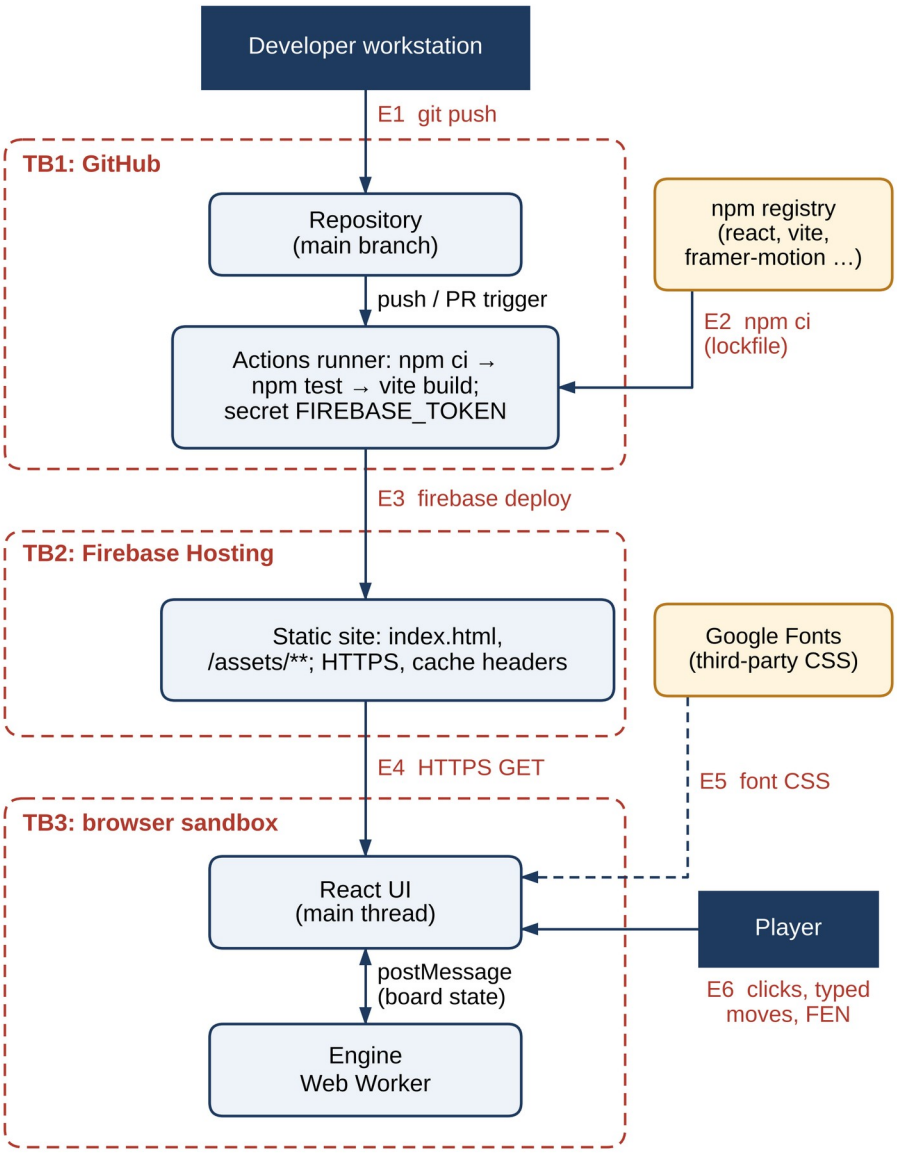


Figure 3.13 Attack Surface and Trust Boundaries

The threat model follows the STRIDE method [17] — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege — applied to the data flows and trust boundaries of the deployed system. The assets to be protected are: the source code and build pipeline (A1); the deployment credential FIREBASE_TOKEN (A2); the deployed static assets (A3); the user's browser session and device resources (A4); and the game state (A5), which has low sensitivity.

Figure 3.13 shows three trust boundaries — GitHub (TB1), Firebase Hosting (TB2) and the user's browser sandbox (TB3) — and six entry points: the developer's git push (E1), dependencies from the npm registry (E2), the deploy step (E3), the HTTPS download of the application (E4), the third-party font stylesheet (E5) and the player's own input (E6).

Table 3.4 lists the threats identified at each entry point, with an estimated risk and the corresponding mitigation. Since the Review 1 prototype, three mitigations have been added: an automated test suite now gates every deployment (T2, T12), FEN entry is validated before use (T10), and the transposition table has a fixed size (T9). Two findings remain from the project's configuration: the deploy step in the GitHub Actions workflow also runs on pull-request events (T4), and no Content-Security-Policy header is set (T7). The other residual risks are typical of static web applications — the software supply chain (T2) and the deployment credential (T3).

Table 3.4 STRIDE Threat Analysis

ID	Entry point / asset	STRIDE	Threat	Risk	Mitigation
T1	E1 – repository (A1)	Tampering	A malicious or accidental commit to main is deployed automatically	Medium	Branch protection with required review (recommended)
T2	E2 – npm dependencies (A1, A3)	Tampering, Elevation	A compromised package runs code at build time or ships in the bundle	Medium	npm ci with committed lockfile; tests before build (in place); Dependabot / npm audit (recommended)

ID	Entry point / asset	STRIDE	Threat	Risk	Mitigation
T3	E3 – CI secret (A2)	Information disclosure	A leaked FIREBASE_TOKEN lets anyone publish to the site	Medium	Encrypted secret, masked in logs (in place); scoped service account (recommended)
T4	E3 – workflow trigger (A3)	Tampering, Elevation	The deploy step also runs on pull_request, so an unmerged branch in the repository could reach production	Medium	Deploy only on push to main (recommended)
T5	E4 – transport (A3)	Tampering, Spoofing	Man-in-the-middle modification of the application	Low	HTTPS-only delivery by Firebase Hosting (in place)
T6	E4 – browser cache (A3)	Tampering	Users keep running an outdated build	Low	no-cache HTML and immutable fingerprinted assets (in place)
T7	E5 – third-party fonts (A4)	Information disclosure	The font request reveals the visitor's IP address; no CSP limits what the page may load	Low	Self-host fonts; add a Content-Security-Policy (recommended)
T8	E6 – user input (A4)	Tampering	Script injection through rendered content	Very low	No user HTML; React escaping; no innerHTML or eval (in place)
T9	E6 – engine worker (A4)	Denial of service	Expensive searches exhaust CPU or memory and freeze the page	Low	Web Worker, time budget, 60 s teaching cap, fixed 10 MB table (in place)
T10	E6 – FEN entry (A5)	Tampering	A malformed or impossible FEN corrupts the engine state	Low	fen.js validates syntax and legality and explains errors (in place)
T11	E6 – typed moves (A5)	Tampering	Malformed or ambiguous typed input is applied as a move	Low	san.js parses and matches against legal moves; refusals are explained (in place)

ID	Entry point / asset	STRIDE	Threat	Risk	Mitigation
T12	Pipeline (A3)	Tampering (integrity)	A regression in the engine or rules reaches users unnoticed	Low	53 tests, including perft, gate every deploy (in place)
T13	Pipeline (A1)	Repudiation	Changes to the live site cannot be traced to an author	Low	Git history and Actions run logs form an audit trail (in place)

3.5 SECURITY CONTROLS

Table 3.5 groups the security controls by layer and shows which are already implemented in the prototype and which are recommended for the next phase, together with the threats from Table 3.4 that each control addresses.

Table 3.5 Security Controls

Layer	Control	Status	Threats
Source	Git history with GitHub as the single source of truth	Implemented	T13
Source	Branch protection and required review on main	Recommended	T1
Supply chain	npm ci installs exactly the committed package-lock.json	Implemented	T2
CI/CD	Automated test suite (53 tests, perft included) must pass before build and deploy	Implemented	T2, T12
Supply chain	Dependabot alerts or npm audit in CI; Actions pinned to commit SHAs	Recommended	T2
CI/CD	Deployment credential kept as an encrypted GitHub secret	Implemented	T3
CI/CD	Deploy only on push to main, with a least-privilege service account	Recommended	T3, T4
Hosting	HTTPS-only delivery	Implemented	T5
Hosting	no-cache HTML; immutable, fingerprinted assets	Implemented	T6
Hosting	Content-Security-Policy, X-Content-Type-Options and Referrer-Policy headers	Recommended	T7, T8
Client	React output escaping; no innerHTML or eval	Implemented	T8

Layer	Control	Status	Threats
Client	Web Worker isolation, time budget, 60 s teaching cap, fixed-size 10 MB table	Implemented	T9
Client	Legal-move validation of clicked and typed moves; FEN validation with explained errors	Implemented	T10, T11
Client	Self-hosted fonts	Recommended	T7
Privacy	No backend, accounts, cookies or analytics	Implemented	—

Two recommended controls can be applied with small configuration changes. The first restricts the deploy step so that it runs only for pushes to the main branch, keeping pull-request runs as build checks [18]:

```
      - name: Deploy to Firebase Hosting
        if: github.event_name == 'push' && github.ref ==
            'refs/heads/main'
        env:
          FIREBASE_TOKEN: ${{ secrets.FIREBASE_TOKEN }}
        run: npx firebase-tools deploy --only hosting
            --project foai-chess-engine --token
            "$FIREBASE_TOKEN"
```

The second adds security headers to firebase.json [19], [21]. The suggested policy allows only the application's own scripts and worker, plus the Google Fonts stylesheet and font files it currently loads; it should be tested on a Firebase preview channel before it is applied to production:

```
{ "source": "**", "headers": [
  { "key": "Content-Security-Policy", "value":
    "default-src 'self'; script-src 'self'; worker-src
        'self';
     style-src 'self' https://fonts.googleapis.com;
     font-src 'self' https://fonts.gstatic.com; img-src
         'self' data:;
     object-src 'none'; base-uri 'self'; frame-ancestors
         'none'" },
  { "key": "X-Content-Type-Options", "value": "nosniff" },
  { "key": "Referrer-Policy", "value":
      "strict-origin-when-cross-origin" } ] }
```

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

Unlike a machine-learning system, this engine is not trained on any dataset. All of its knowledge is either the rules of chess, encoded in the move generator, or hand-crafted heuristics in the evaluation function. The data it works with, and the data collected to evaluate it, fall into four groups:

- **Game-state data:** the current position — piece placement, side to move, castling rights, en passant square and move clocks — supplied by the interface as a serialised board, together with the list of position keys played so far, the contempt setting and the time budget.
- **Reference data:** the published perft (performance test) node counts for the seven standard test positions [11] (Table 4.1), used to verify move generation, and the standard material values and piece-square tables of the Simplified Evaluation Function [12], used as the base of the evaluation.
- **Test positions:** 17 experiment positions — four openings, four busy middlegames, five tactical positions including the Opera Game mate and Win at Chess #1 and #4 [23], and four endgames including Fine #70 [24]; 40 book openings for matches; and a 105-position tactical suite built from weak Stockfish games, with every position verified by Stockfish at depth 16 and kept only when one move is clearly best.
- **Measurement data:** node counts, times, depths, moves and scores from 917 teaching-search runs; the games of the self-play matches and of the 128-game benchmark against Stockfish (saved in PGN); and the engine console's live telemetry.

Perft (performance test) counts every leaf of the legal-move tree to a given depth. Because the values for the standard positions are known exactly, any difference exposes a bug in move generation — a missing en passant capture, castling through check, a pinned piece allowed to move or a dropped promotion. Together the seven positions below exercise all of these, and the mirrored position 4 catches asymmetries between White and Black.

Table 4.1 Performance Test (Perft) Reference Values for the Standard Test Positions

Position	d1	d2	d3	d4	d5
Start	20	400	8,902	197,281	4,865,609
Kiwipete (2)	48	2,039	97,862	4,085,603	—
Position 3	14	191	2,812	43,238	674,624
Position 4	6	264	9,467	422,333	—
Position 4, mirrored	6	264	9,467	422,333	—
Position 5	44	1,486	62,379	2,103,487	—
Position 6	46	2,079	89,890	3,894,594	—

4.2 DATA PREPROCESSING

Before the search can use it, raw game data is converted into forms the engine can process efficiently and safely:

- **Input validation:** a FEN string is checked by fen.js before it is loaded, and a typed move is parsed by san.js and matched against the legal-move list; invalid input is refused with an explanation.
- **State encoding:** a FEN string is parsed into the 64-square array, using the index rank × 8 + file with rank 0 as the eighth rank, and into the side-to-move, castling, en passant and clock fields.
- **Move encoding:** moves are held as objects carrying everything needed to undo them, written in SAN for the move list and console and in UCI for messages, and packed into a single integer (from, to, promotion) for the transposition table.
- **Snapshot serialisation:** serialize() turns the board into a plain object that is copied into the worker by postMessage; loadFrom() rebuilds an independent board inside the worker.
- **Position hashing:** 64-bit Zobrist keys are held as two 32-bit halves, so the exclusive-OR is exact without allocating a BigInt at every node; the random keys come from the Mulberry32 generator with a fixed seed. Repetition detection uses an exact string key so that no collision is possible.
- **Score normalisation:** evaluation is in centipawns from White's view; Negamax flips the sign for the side to move; piece-square tables are read at index 63 − sq for Black;

mate scores are ±20,000 adjusted by distance and stored in the table relative to the node; and the evaluation bar clamps to ±8 pawns, showing forced mates as M<n>.

The evaluation function combines the terms in Table 4.2 into a single score E(s) = Material + Piece-square + Pawn structure + King safety + Mobility.

Table 4.2 Evaluation Function Terms

Term	Rule	Values (centipawns)
Material	Sum of piece values for White minus Black	P 100, N 320, B 330, R 500, Q 900, K 20,000
Piece-square	Per-piece 64-entry tables, mirrored for Black	Between −50 and +50 per piece
Pawn structure	Pawns on a file with two or more own pawns; isolated pawns	−15 per doubled pawn; −20 per isolated pawn
King safety	No own pawn on the king's file; also no enemy pawn; pawn shield when the king is on files a–c or f–h	−30; a further −20; +10 per shield pawn
Mobility	Knight moves (precomputed per square) and bishop moves	+2 per available move

4.2.1 ALGORITHMS / METHODOLOGY

The engine combines the algorithms below. They are given in pseudo-code that follows the implementation.

4.2.1.1 Pseudo-code of the Algorithms

Algorithm 1: Iterative Deepening (getBestMove)

Input: board position; time budget T (2,000 ms, or derived from the clock)
Output: best move for the side to move

```
1:  start ← now();  best ← none;  TT.NEWSEARCH()
2:  for depth ← 1 to 12 do
3:      if now() − start ≥ T then break
4:      (score, move) ← ROOT(depth, −∞, +∞)
5:      if the search was aborted then break        ▷ keep the last completed result
6:      best ← move;  report(depth, score, nodes)
7:      if |score| > 15,000 then break              ▷ forced mate found
8:  end for
9:  return best
```

Algorithm 2: Negamax with Alpha-Beta, Null Move and Transposition Table

Input: depth, ply, search window (α, β), allowNull
Output: value of the position for the side to move

```
count the node; abort if the time budget is spent
if the position repeats or halfMoveClock ≥ 100 then return −contempt        ▷ draw
key ← ZOBRIST(board)
(value, ttMove) ← TT.LOOKUP(key, depth, α, β, ply)
if value ≠ null then return value
if depth ≤ 0 then return QUIESCENCE(α, β)
moves ← LEGALMOVES(board)
if moves = ∅ then return (in check ? −20,000 + ply : −contempt)
if allowNull and not in check and depth ≥ 4 and |β| < 15,000 and the side to move
    has pieces other than pawns then
    v ← −NULLSEARCH(depth − 1 − 2, ply + 1, β)        ▷ R = 2, null window
    if v ≥ β then return β        ▷ prune the node
end if
α₀ ← α;  best ← −∞
for each m in ORDER(moves, ttMove) do
    MAKE(m);  v ← −NEGAMAX(depth − 1, ply + 1, −β, −α, true);  UNMAKE(m)
    if v > best then best ← v;  bestMove ← m
    α ← max(α, v)
    if α ≥ β then break        ▷ β cut-off
end for
flag ← UPPER if best ≤ α₀;  LOWER if best ≥ β;  otherwise EXACT
TT.STORE(key, depth, best, flag, bestMove, ply)
return best
```

Algorithm 3: Quiescence Search

Input: search window (α, β)
Output: value of a quiet position for the side to move

```
standPat ← EVALUATE(board) × (White to move ? 1 : −1)
if standPat ≥ β then return β
α ← max(α, standPat)
for each capture c in ORDER(CAPTURES(board)) do
    MAKE(c);  v ← −QUIESCENCE(−β, −α);  UNMAKE(c)
    if v ≥ β then return β
    α ← max(α, v)
end for
return α
```

Algorithm 4: Move Ordering (MVV-LVA)

Input: move m; hash move ttMove from the transposition table
Output: ordering score s (moves are searched in descending order of s); value: P 1, N 3, B 3, R 5, Q 9, K 100

```
1: s ← 0
2: if m = ttMove then s ← 10,000
3: if m captures victim V with attacker A then s ← s + 100 + 10·value(V) − value(A)
4: if m is a promotion then s ← s + 50
5: return s
```

Algorithm 5: Zobrist Hashing and Table Replacement

Input: board position (keys from Mulberry32 with the fixed seed 42)
Output: 64-bit key held as two 32-bit halves (hi, lo); updated table slot

```
1: (hi, lo) ← (0, 0)
2: for each occupied square sq holding piece p do (hi, lo) ← (hi, lo) XOR Z[p][sq]
3: if Black to move then (hi, lo) ← (hi, lo) XOR Z_side
4: (hi, lo) ← (hi, lo) XOR Z_castle[rights]        ▷ 16 combinations
5: if an en passant square exists on file f then (hi, lo) ← (hi, lo) XOR Z_ep[f]
6: return (hi, lo)
   procedure STORE(key, depth, score, flag, move, ply)        ▷ 2^19 slots (10 MB)
7:     i ← lo(key) AND (2^19 − 1)
8:     replace slot i if it holds the same position, or it is from an earlier search, or depth ≥ stored depth
9:     save the key halves, score (mates relative to the node), move, depth, flag and age
```

4.2.1.2 Complexity

For branching factor b and depth d, Minimax takes $O(b^d)$ time and O(bd) memory. With perfect ordering, Alpha-Beta examines about $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves [3], roughly doubling the reachable depth — a per-ply cost near $\sqrt{b}$. Iterative Deepening repeats the shallower iterations, but because each level is about b times larger than the one before it, the extra work is a factor of only about $b / (b - 1)$ [4], while it provides a move at any moment and good ordering information from the previous iteration. The measured effective branching factors in Section 4.2.2 confirm this behaviour.

4.2.1.3 Experimental Methodology

Search techniques were compared with the teaching search — the same code as the teaching-mode screen — on the 17 experiment positions at every depth from 1 to 7 under eight configurations, from plain Minimax to everything switched on. A configuration stopped going deeper on a position once a search passed 30 seconds; 917 searches ran, 51 of which hit that limit. Node counts are exact and deterministic; times were measured on one core (Node 22, Linux). Playing strength was measured in games: self-play matches of 80 games at 300 ms per move over 40 book openings with both colours, and 128 games at one second per move against Stockfish 19 with its strength limited to calibrated UCI_Elo settings [10]. Ratings were estimated by maximum likelihood with the logistic Elo model [22]. The search parameters are listed in Table A.1 of the Appendix.

4.2.2 RESULTS AND DISCUSSION

4.2.2.1 Correctness

The move generator matches every published perft count in Table 4.1 — 30 depths across seven positions, 17.0 million leaves in all. These checks run automatically: Table A.2 in the Appendix lists the 53 tests that run in about four seconds on every build, and CI refuses to deploy if any of them fails. The full suite was re-run while preparing this report, and all 53 tests passed.

4.2.2.2 Start Position at Depth 5

Table 4.3 and Figure 4.1 show the teaching-mode comparison on the starting position. Plain Minimax visits 5,072,213 nodes — exactly 1 + 20 + 400 + 8,902 + 197,281 + 4,865,609, the cumulative perft count — which confirms the move generator and the search together. These node counts were reproduced exactly in the deployed interface while preparing this report (Figure 4.5).

Table 4.3 Search Configuration Comparison (Starting Position, Depth 5)

Configuration	Nodes searched	Time (s)	Reduction	Move chosen
Plain Minimax	5,072,213	15.0	baseline	e4
+ Alpha-Beta	113,360	0.42	45× fewer	e4
+ MVV-LVA + TT	58,390	0.28	87× fewer	e4

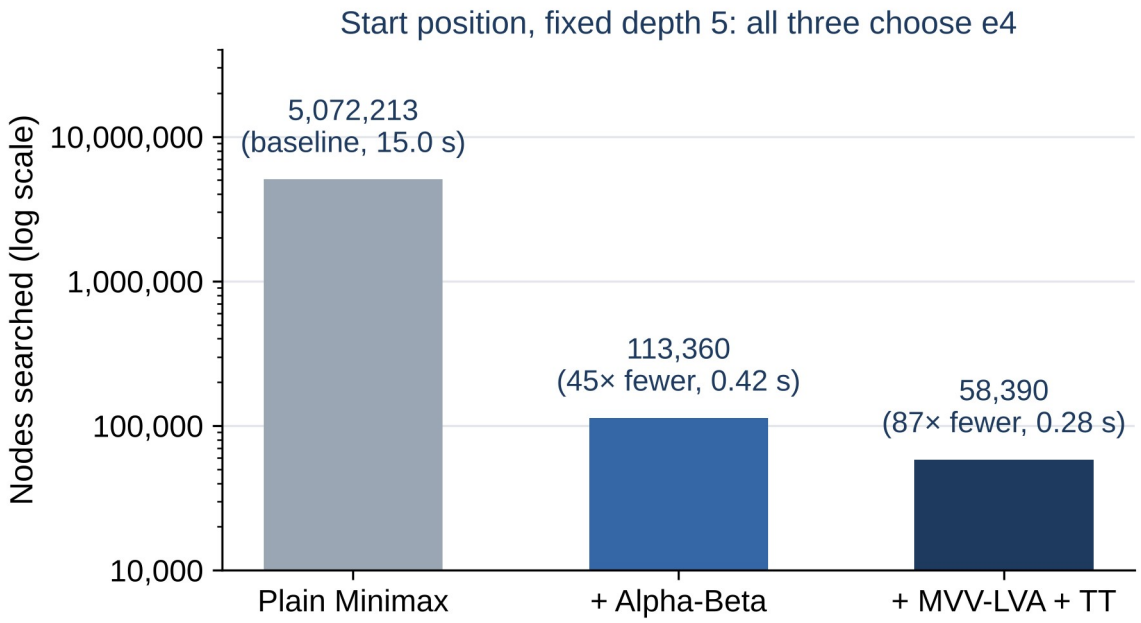


Figure 4.1 Nodes Searched by Configuration on the Starting Position

4.2.2.3 Across 17 Positions and Depths 1–7

Table 4.4 gives the reduction in nodes relative to plain Minimax as a geometric mean over the positions where Minimax finished (all 17 up to depth 4), and Figure 4.2 plots the first four depths. Alpha-Beta alone saves little at shallow depths but about 20-fold by depth 4. Move ordering is what makes it pay: at depth 4 it multiplies the saving about five times, to roughly 100-fold, because Alpha-Beta can only cut a branch once it has seen a good enough move, and trying captures of valuable pieces first finds that move early. On Kiwipete at depth 5, Alpha-Beta alone needs 6.6 million nodes and with ordering 75 thousand — 89 times fewer. The transposition table adds little on its own at shallow depth and more with every ply, as more move orders reach the same positions; its stored best move also improves ordering.

Table 4.4 Node Reduction Relative to Plain Minimax (Geometric Mean)

Configuration	d2	d3	d4	d5	d6	d7
Alpha-Beta	2.6×	5.8×	20.8×	17.5×	20.6×	28.5×
+ MVV-LVA ordering	6.2×	17.3×	102.6×	48.4×	51.4×	31.0×
+ TT	2.6×	5.8×	23.2×	26.3×	47.5×	169.5×
+ ordering + TT	6.2×	17.3×	110.6×	70.1×	105.7×	187.2×
+ ordering + TT + null move	6.2×	17.3×	110.6×	105.1×	116.3×	187.2×
Positions averaged	17	17	17	6	3	2

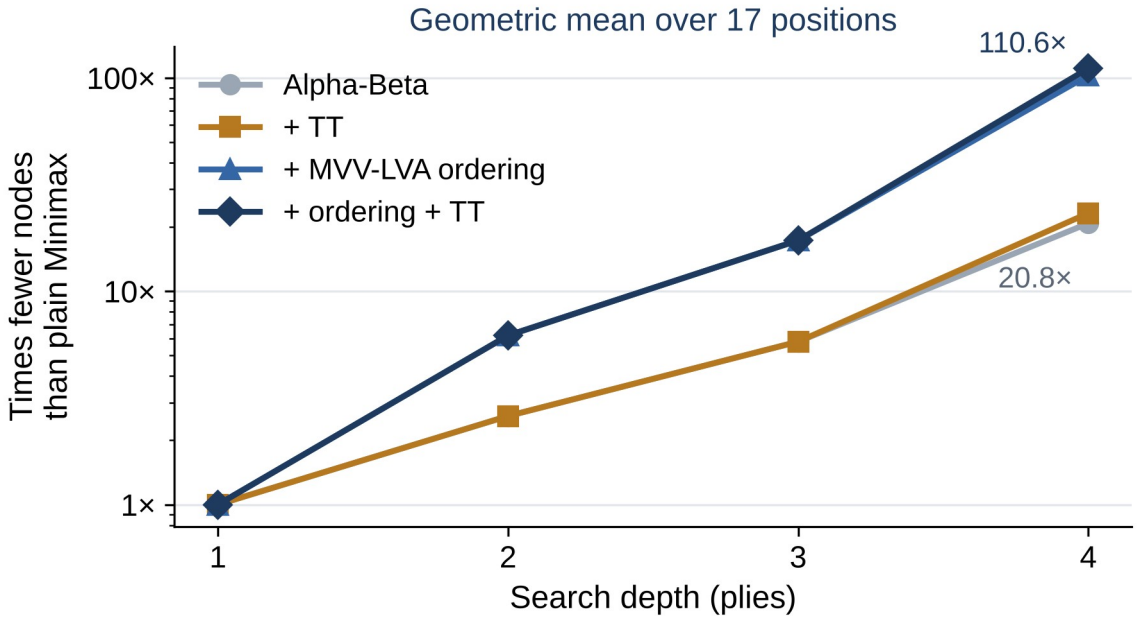


Figure 4.2 Node Reduction by Depth (Geometric Mean over 17 Positions)

Table A.3 in the Appendix shows the effective branching factor — how many times more nodes each extra ply costs. Plain Minimax pays the full branching factor of chess, 24 to 27 in these positions. With ordering, the table and null-move pruning, a ply costs only 2 to 5 times more from depth 4 on, close to $\sqrt{b}$, which is what Alpha-Beta with perfect ordering would give. The alternating pattern in the Alpha-Beta row is the odd–even effect predicted by the $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor}$ formula.

4.2.2.4 Same Answer, Less Work

For every position and depth where plain Minimax finished — 79 searches — the other configurations were compared with it (Table 4.5). Alpha-Beta, move ordering and the transposition table returned exactly Minimax's score and move in all 79, which is the equivalence that teaching mode claims. With the four-ply guard, null-move pruning also agreed in all 79, although nothing guarantees that it will. Quiescence answers a different question: it keeps searching captures past the fixed depth, so its scores differ in most positions and its move in a third of them — which is the point of it.

Table 4.5 Agreement with Plain Minimax (79 Searches)

Configuration	Same score	Same move
Alpha-Beta	79 / 79	79 / 79
+ MVV-LVA ordering	79 / 79	79 / 79

Configuration	Same score	Same move
+ TT	79 / 79	79 / 79
+ ordering + TT	79 / 79	79 / 79
+ ordering + TT + null move	79 / 79	79 / 79
+ ordering + TT + quiescence (and everything)	31 / 79	52 / 79

4.2.2.5 Speed per Node

Fewer nodes do not always mean less time. Move ordering costs over 40 % of the speed per node, because every node sorts its moves, and is still by far the best trade. The transposition table costs about 7 % per node, since hashing is two 32-bit exclusive-ORs per piece. On the starting position at depth 5, ordering and the table together take 0.28 s, against 0.42 s for Alpha-Beta alone and 15.0 s for plain Minimax. An earlier optimisation of the evaluation function, which had taken about 65 % of search time, cut depth-5 plain Minimax from 110.2 s to 13.6 s with identical results.

4.2.2.6 Null-Move Pruning

The first version of null-move pruning tried a null move whenever more than two plies remained, so the reduced search could fall straight into quiescence. On Win at Chess #1 (1.Qg6! gxf6 2.Qh7#), that capture-only search could not see the quiet threat Qh7#, cut Black's replies to 1.Qg6, and the engine played 1.Ne8 instead, needing depth 6 to find a mate it found at depth 4 without null move. Requiring at least four plies at the node keeps every null search one full ply deep, and the mate reappears at depth 4. Whether this guard costs strength was settled by games (Table 4.6): null-move pruning is worth well over 100 Elo, and the stricter guard's cost is within the noise.

Table 4.6 Self-Play Matches (80 Games Each, 300 ms per Move)

Match (first engine's result)	+	=	−	Score	Elo difference (95 %)
Null move (3-ply guard) vs no null move	49	11	20	68.1 %	+132 (+61 to +215)
Null move, 4-ply guard vs 3-ply guard	28	17	35	45.6 %	−30 (−100 to +37)
Final engine vs engine before this iteration	42	17	21	63.1 %	+93 (+27 to +168)

On the 105-position tactical suite (Table A.4 in the Appendix), the builds differ far less than in games. Most positions are "only moves" that any sufficiently deep search finds, so time matters most: from 78 to 94 solved as the budget rises from 0.2 to 5 seconds. Null move buys about half a ply at 5 s (7.27 against 6.81) and one more position solved. Against the engine as it stood before this iteration, the final engine also scored 63 % (+93 Elo), searching 6.41 plies deep on average against 5.83 at the same time per move.

4.2.2.7 Fixed-Size Transposition Table

The table was previously a JavaScript Map that grew for as long as a session ran; it is now a fixed 10 MB structure kept for the whole game. Making it fixed-size exposed a flaw: the Zobrist keys came from a linear congruential generator whose low bits are nearly periodic — bit 0 simply alternates — and the new table chooses a slot by exactly those low bits, so half the slots could never be used. The keys now come from Mulberry32, and a test requires every slot of a small table to be reachable. In teaching mode the new table is invisible, as it should be: on the start position it searches 58,390 nodes against the Map's 58,356, with the same move and score. Table A.5 in the Appendix shows that the shipped size sits on the flat part of the curve.

The table can decide whether a position is solvable at all. In Fine #70, 1.Kb1 wins only because White's king can break through about twenty plies deep. With an effectively empty table the engine does not get past depth 16 in a minute; with any real table it plays 1.Kb1 within 0.4 to 14 seconds, because the many move orders of the king manoeuvres collapse into far fewer positions.

4.2.2.8 Contempt

A draw found inside the search is worth −contempt to the engine. Tests confirm the behaviour on positions where the choice is stark: after 1.Nf3 Nf6 2.Ng1, at contempt −300 the engine returns to the starting position with 2…Ng8, and at +300 it plays anything else; with a lone queen against a cornered king, at contempt −2,000 it stalemates on purpose and at 0 it keeps the win. Contempt changes only the value of draws, so it has no cost where no draw is in reach.

4.2.2.9 Playing Strength

The reference engine was Stockfish 19 with UCI_LimitStrength enabled, whose UCI_Elo settings are calibrated by its developers against rated engines. Both engines had one

second per move; each opponent played 32 games over 16 book openings with both colours; games were adjudicated by the project's own rules code. A pilot of six games against the lowest setting, 1320, was won 6–0, so the benchmark used 1500, 1700, 1900 and 2100 (Table 4.7).

Table 4.7 Results against Stockfish 19 at Calibrated Strengths (1 s per Move)

Opponent (UCI_Elo)	Games	+	=	−	Score	Implied rating (95 %)	Mean depth
1500	32	21	0	11	65.6 %	1612 (1494–1764)	6.34
1700	32	11	0	21	34.4 %	1588 (1436–1706)	6.39
1900	32	9	1	22	29.7 %	1750 (1587–1867)	6.46
2100	32	8	2	22	28.1 %	1937 (1772–2052)	6.65

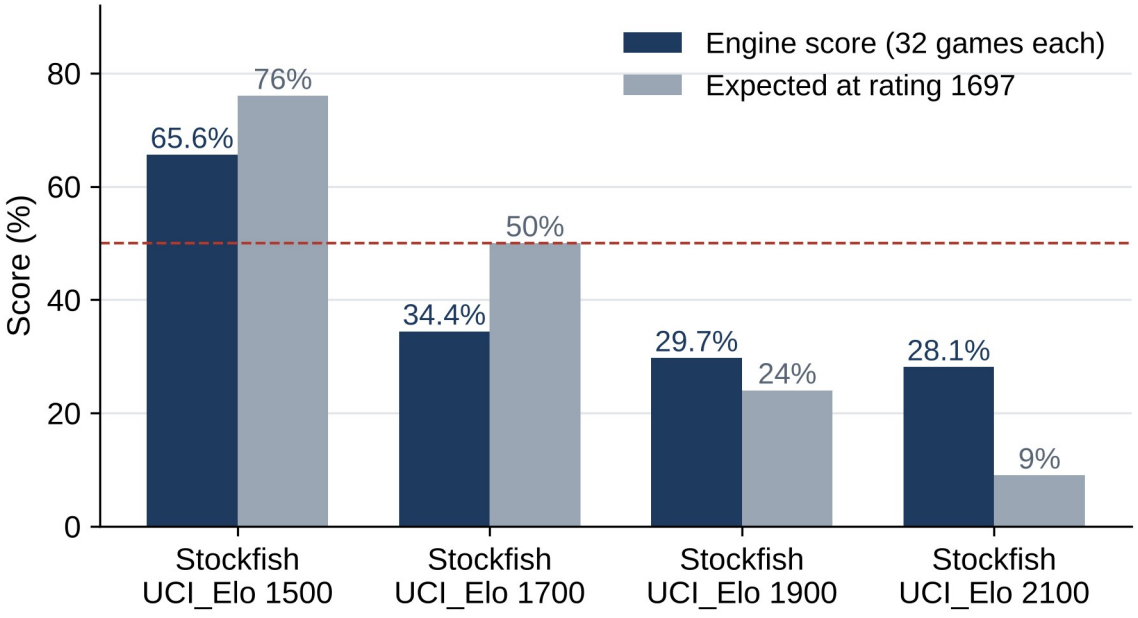


Figure 4.3 Score against Each Stockfish Setting versus the Score Expected at 1697

The pooled maximum-likelihood rating is 1697 ± 72, and the two anchors that bracket the engine give 1600 ± 89. The estimates differ because the results do not follow one logistic curve (Figure 4.3): the stronger settings lose to the engine far more often than their ratings predict, as strength-limited Stockfish plays like full-strength Stockfish that occasionally picks a worse move deliberately, and an engine that converts such gifts at six or seven plies wins those games. The honest statement is therefore a range — about 1600 to 1700 on Stockfish's limited-strength scale at one second per move, with 1600 the better-supported end. Of the 128

games, 125 ended in checkmate, and colour made no difference (25 points as White, 25½ as Black).

4.2.2.10 Working Prototype

Figures 4.4 to 4.6 show the deployed prototype after these changes: the play screen in a rapid game, with clocks, the SAN move history and the engine console; teaching mode on the starting position, with the FEN row and the five presets; and the new-game dialog.



Figure 4.4 Prototype – Play Screen with Clocks and SAN History

In Figure 4.4 the player has White in a rapid 10+0 game. After 1.e4 d5 2.exd5 Nf6 3.Nc3 Nxd5 4.Nf3 Nxc3 5.dxc3 Qxd1+ the engine has just exchanged queens with check: the last move is highlighted on the board and in the SAN move history, the king in check is marked in red, and the captured pieces are listed beside each player. Both clocks are shown, and the box below the board accepts the reply typed in SAN or UCI. The engine console on the right reports the search that produced Qxd1+ — the best move, the depth reached, the

number of nodes and the score — together with one line for every completed iteration and how full the transposition table is.

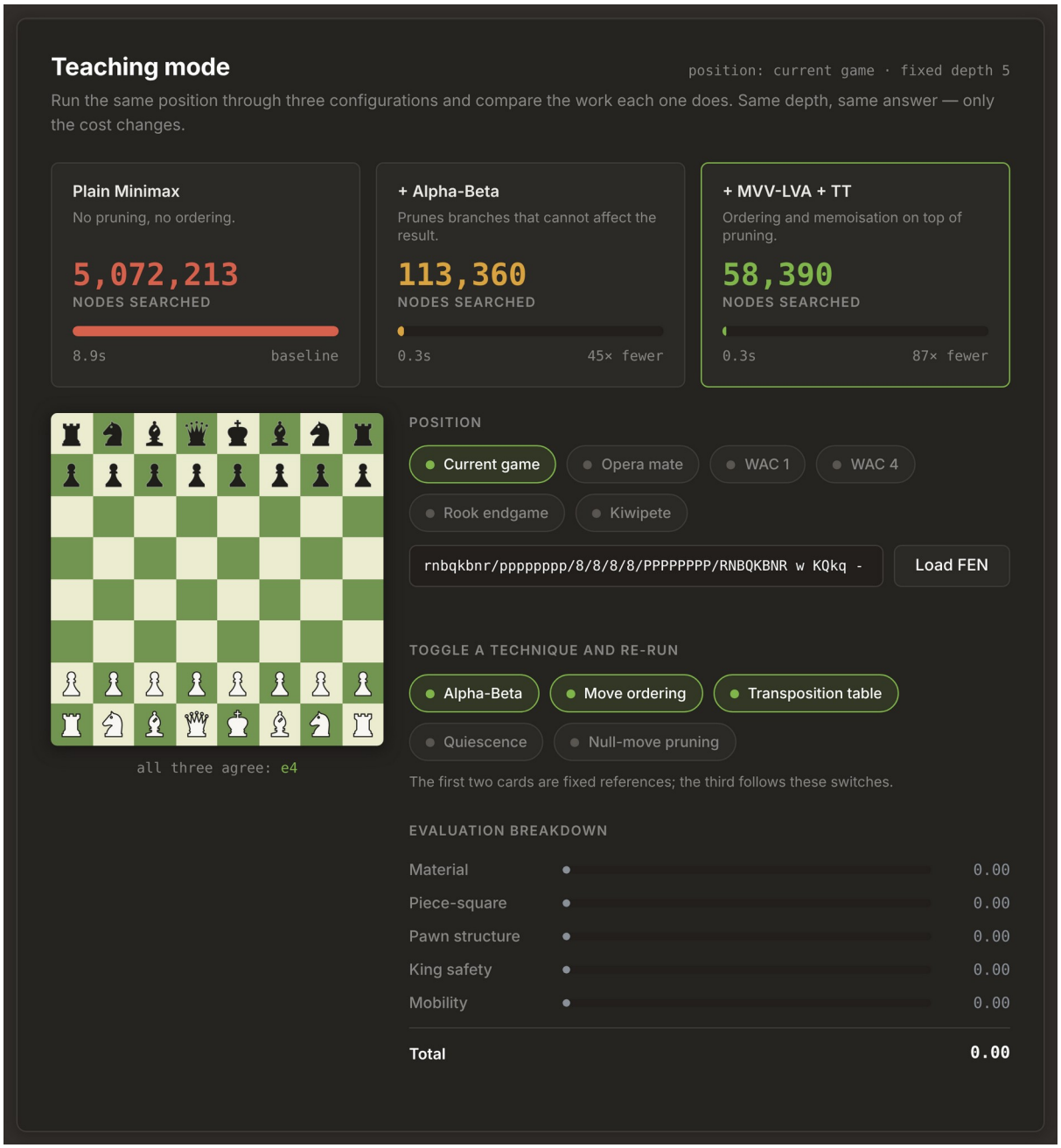


Figure 4.5 Prototype – Teaching Mode with FEN Presets

Figure 4.5 shows teaching mode on the starting position at the fixed depth of five. The three cards report the work done by each configuration: plain Minimax searches 5,072,213 nodes, Alpha-Beta 113,360 (45 times fewer) and Alpha-Beta with MVV-LVA ordering and the transposition table 58,390 (87 times fewer), and all three choose the same move, 1.e4. Below the cards, the player can switch to one of the five preset positions or paste any FEN

string, switch each technique on or off for the third card, and read the five-term evaluation breakdown, which is exactly zero for the symmetrical starting position.

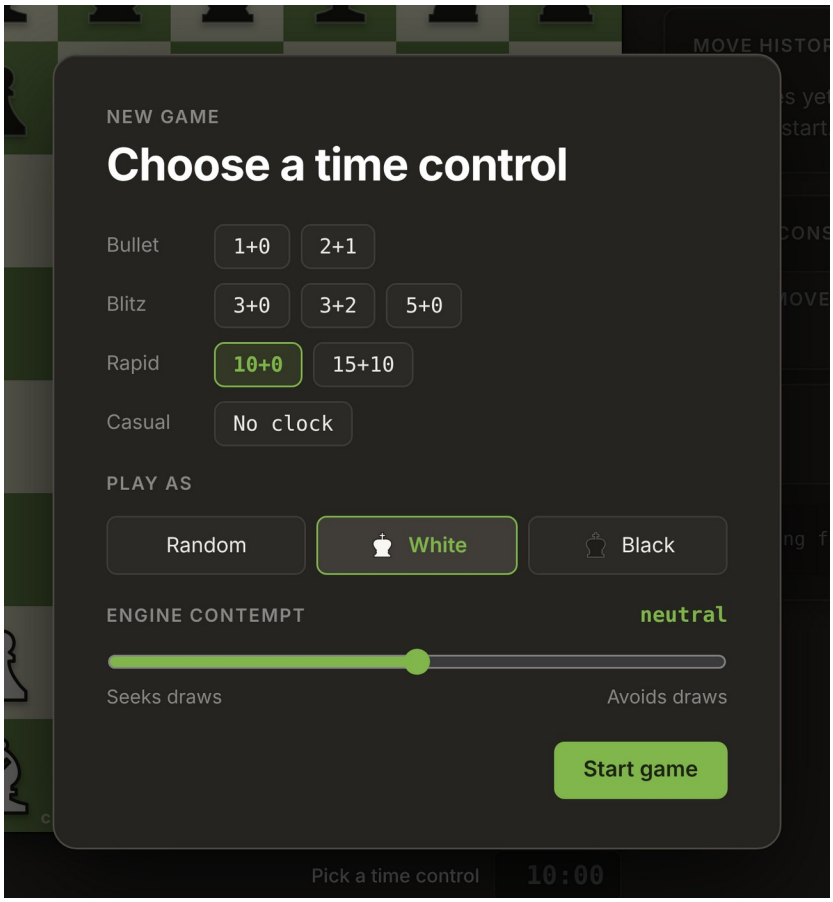


Figure 4.6 Prototype – New-Game Dialog

Figure 4.6 shows the dialog that opens every match. The player picks one of eight time controls in four groups — bullet (1+0, 2+1), blitz (3+0, 3+2, 5+0), rapid (10+0, 15+10) or a casual game with no clock — chooses to play White, Black or a random colour, and sets the engine's contempt with a slider that runs from "seeks draws" to "avoids draws". The selected options are outlined in green, and the board remains visible behind the dialog.

4.2.2.11 Limitations Observed

- Quiescence ignores checks: a leaf in check is scored by standing pat, so a quiet mate at the horizon is invisible. This is what forced the four-ply null-move guard.
- Move ordering stops at captures and promotions; quiet moves are searched in generation order, and checks are not extended.
- A draw score stored because one line repeated can be reused on a path where it did not; this is bounded by clearing the table when contempt or side changes.

- When the time budget expires part-way through an iteration, the console reports the depth of that unfinished iteration (for example depth 8 when the log ends at depth 7); the move played is still from the last completed iteration.
- The Elo figure has a defined scope: one reference engine family, one time control and 128 games.

CHAPTER 5

CONCLUSION

5.1 SUMMARY OF THE WORK

This project set out to show that a transparent, classical AI agent can play chess correctly, efficiently and measurably well. Chess was formulated as a fully observable, deterministic, zero-sum state space, and a goal-based agent was built from first principles: a self-written board representation, legal move generator, FEN validator and SAN parser; a five-term hand-crafted evaluation function; and Negamax search with Alpha-Beta pruning, Iterative Deepening, Quiescence Search, MVV-LVA move ordering, null-move pruning, configurable contempt and a fixed-size Zobrist-hashed transposition table. No machine learning, training data or external chess library is used.

The engine is correct where it can be checked exactly: it matches the published performance test (perft) counts on all seven standard positions, and Alpha-Beta, ordering and the transposition table agree with plain Minimax in all 79 comparable searches, with 53 automated tests guarding every deployment. The experiments make the course's central point in numbers: plain Minimax pays the full branching factor of chess, 24 to 27 per ply; Alpha-Beta alone brings that down to 7 to 12; and move ordering turns Alpha-Beta's theoretical saving into a real one, about 100-fold at depth 4. Null-move pruning is the one technique that trades exactness for depth, worth about 130 Elo in games, and a careless guard for it missed a mate in two. The final engine beats its earlier version by about 93 Elo and plays at roughly 1600–1700 on Stockfish's calibrated scale at one second per move.

The project also treats its own delivery as a security problem. A STRIDE analysis of the build pipeline, hosting and client identified the supply chain and the deployment pipeline as the main residual risks. Since the first review, automated tests have been added to gate every deployment, FEN and typed moves are validated, and the transposition table's memory has been bounded; the remaining recommendations are to deploy only on pushes to the main branch, add a Content-Security-Policy and related headers, use a least-privilege deployment credential and automate dependency checks.

5.2 FUTURE WORK

- Search check evasions in quiescence, removing the blind spot that forced the null-move guard.
- Order quiet moves with killer moves and the history heuristic, and add check extensions.
- Update the Zobrist key incrementally in make and unmake instead of recomputing it at every node.
- Report the depth of the last completed iteration in the engine console.
- Apply the recommended security controls: deploy-on-push only, security headers, a scoped service account, pinned Actions, dependency scanning and self-hosted fonts.
- Widen the strength benchmark to other reference engines and time controls.

REFERENCES

[1] C. E. Shannon, "Programming a computer for playing chess," Philosophical Magazine, ser. 7, vol. 41, no. 314, pp. 256–275, 1950.

[2] A. L. Zobrist, "A new hashing method with application for game playing," Tech. Rep. 88, Computer Sciences Department, University of Wisconsin, Madison, 1970.

[3] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," Artificial Intelligence, vol. 6, no. 4, pp. 293–326, 1975.

[4] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," Artificial Intelligence, vol. 27, no. 1, pp. 97–109, 1985.

[5] M. Campbell, A. J. Hoane Jr., and F. Hsu, "Deep Blue," Artificial Intelligence, vol. 134, no. 1–2, pp. 57–83, 2002.

[6] R. Coulom, "Efficient selectivity and backup operators in Monte-Carlo tree search," in Computers and Games (CG 2006), LNCS vol. 4630, Springer, 2007, pp. 72–83.

[7] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020, ch. 5, "Adversarial Search and Games."

[8] D. Silver et al., "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," Science, vol. 362, no. 6419, pp. 1140–1144, 2018.

[9] Y. Nasu, "Efficiently updatable neural-network-based evaluation functions for computer shogi," The 28th World Computer Shogi Championship Appeal Document, 2018.

[10] The Stockfish developers, "Stockfish – open-source chess engine." [Online]. Available: https://stockfishchess.org

[11] Chess Programming Wiki, "Perft results." [Online]. Available: https://www.chessprogramming.org/Perft_Results

[12] T. Michniewski, "Simplified evaluation function," Chess Programming Wiki. [Online]. Available: https://www.chessprogramming.org/Simplified_Evaluation_Function

[13] T. A. Marsland, "A review of game-tree pruning," ICCA Journal, vol. 9, no. 1, pp. 3–19, 1986.

[14] D. F. Beal, "A generalised quiescence search algorithm," Artificial Intelligence, vol. 43, no. 1, pp. 85–98, 1990.

[15] J. Schaeffer, "The history heuristic and alpha-beta search enhancements in practice," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 11, no. 11, pp. 1203–1212, 1989.

[16] C. Donninger, "Null move and deep search: Selective-search heuristics for obtuse chess programs," ICCA Journal, vol. 16, no. 3, pp. 137–143, 1993.

[17] A. Shostack, Threat Modeling: Designing for Security. Wiley, 2014.

[18] GitHub Docs, "Security hardening for GitHub Actions." [Online]. Available: https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions

[19] MDN Web Docs, "Content Security Policy (CSP)." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

[20] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," A/RES/70/1, 2015.

[21] Firebase Documentation, "Configure Hosting behavior." [Online]. Available: https://firebase.google.com/docs/hosting/full-config

[22] A. E. Elo, The Rating of Chessplayers, Past and Present. New York: Arco, 1978.

[23] F. Reinfeld, Win at Chess. New York: Dover, 1958.

[24] R. Fine, Basic Chess Endings. Philadelphia: David McKay, 1941.

APPENDIX

A.1 PROJECT LINKS

- Source code repository: https://github.com/SaIdEeVaN/FOAI-Project
- Live prototype: https://foai-chess-engine.web.app
- Detailed experimental report in the repository: REPORT.md; raw results in bench/results/

A.2 DETAILED EXPERIMENTAL RESULTS

The tables below give the engine parameters and the detailed results referred to in Chapter 4: the parameters used in every experiment (Table A.1), the automated test suite (Table A.2), the effective branching factor of each configuration (Table A.3), the tactical-suite results (Table A.4) and the transposition-table sizing study (Table A.5).

Table A.1 Engine Parameters

Parameter	Value
Iterative deepening depth	1 to 12 plies
Time budget	2,000 ms untimed; with a clock, remaining ÷ 30 + 0.8 × increment, between 50 and 2,000 ms; checked every 2,048 nodes
Mate score	±20,000, adjusted by distance to mate
Early exit	|score| > 15,000 (forced mate found)
Null-move pruning	R = 2; at least 4 plies left; not in check, twice in a row, near mate, or with only king and pawns
Contempt	−100 to +100 centipawns in steps of 10 (default 0)
Transposition table	2^{19} slots, about 10 MB, kept for the whole game
Zobrist keys	64-bit, as two 32-bit halves from Mulberry32 with a fixed seed (42)
Teaching mode	Fixed depth 5; 60 s cap per configuration, then reported as a lower bound
Evaluation bar range	±8 pawns; forced mates shown as M<n>

Table A.2 Automated Test Suite

Test file	What it verifies
perft.test.js	Perft counts on the seven standard positions
fen.test.js	FEN round trips; validation messages for 15 kinds of bad FEN
san.test.js	The Opera Game read and written move for move; en passant, promotions, disambiguation, castling, check and mate marks; forgiving input and refusal reasons
repetition.test.js	Threefold repetition, side to move and castling in the key, en passant only when capturable
evaluation.test.js	Colour symmetry term by term; the breakdown adding up to the total
transposition.test.js	Fixed size, bound and depth rules, full-key checks, replacement, mate scores by ply, every slot reachable
search.test.js	Mates in one and two with and without null move; Win at Chess #1 at depth 4; contempt towards and away from draws
teaching.test.js	Plain Minimax equal to cumulative perft; Alpha-Beta, ordering and TT matching Minimax's score
ui-helpers.test.js	Score and mate formatting; premove target squares

Table A.3 Effective Branching Factor

Configuration	d1→d2	d2→d3	d3→d4	d4→d5	d5→d6	d6→d7
Plain Minimax	23.64	27.33	24.16	14.30*	8.14*	6.59*
Alpha-Beta	9.26	11.99	6.75	10.45	4.04	6.87
+ ordering + TT	3.83	9.72	3.79	6.92	2.16	3.86
+ ordering + TT + null move	3.83	9.72	3.79	5.14	2.11	2.87
Everything (with quiescence)	4.07	6.97	3.08	3.85	3.48	2.78

(* Plain Minimax finished only the smaller endgames at depths 5 to 7, so those columns understate its cost.)

Table A.4 Tactical Suite Results (Positions Solved out of 105)

Build	0.2 s	1 s	5 s	Mean depth (0.2 / 1 / 5 s)
Final engine	78	88	94	4.53 / 5.83 / 7.27
Final engine, null move off	79	88	93	4.50 / 5.76 / 6.81
Before this iteration	77	88	93	4.50 / 5.71 / 6.79

Table A.5 Transposition-Table Size against Search Effort (Nodes at Fixed Depth)

Position (depth)	2^8 slots	2^{12} slots	2^{16} slots	2^{19} slots	2^{22} slots
Start position (7)	2,143,841	1,341,275	953,394	920,884	905,827
Kiwipete (6)	962,788	720,616	613,945	603,633	601,854
Perft position 6 (6)	1,057,614	924,659	833,666	805,468	804,559
Memory	5 KB	78 KB	1.2 MB	10 MB	80 MB

A.3 REPOSITORY STRUCTURE

Table A.6 lists the main folders and files of the repository and what each contains.

Table A.6 Repository Structure

Path	Contents
src/components/	ChessBoard, MoveHistory, EngineConsole, TeachingMode, MoveEntry (typed moves), TimeControl (clocks), Modals, PieceSymbols (SVG pieces)
src/engine/board.js	Board state, make and unmake, FEN in and out
src/engine/moveGen.js	Legal move generation
src/engine/move.js	Move encoding and UCI notation
src/engine/san.js	SAN output and parsing of typed moves
src/engine/fen.js	FEN validation with plain-language errors
src/engine/evaluation.js	Static evaluation and per-term breakdown
src/engine/search.js	Iterative deepening, null move, contempt
src/engine/teachingSearch.js	Fixed-depth search with switchable techniques
src/engine/transposition.js	Zobrist hashing and the fixed-size table
src/engine/premove.js	Premove target squares
src/engine/worker.js	Web Worker entry point
src/App.jsx	Root component and game logic
src/index.css	Design tokens and styles
test/	node:test suite (53 tests)
bench/	Experiments, matches, tactical suite and Elo benchmark
.github/workflows/deploy.yml	Test, build and deploy pipeline
firebase.json	Hosting and cache-header configuration

A.4 SOURCE CODE – NEGAMAX SEARCH WITH NULL MOVE (search.js)

```
_negamax(depth, ply, alpha, beta, allowNull) {
  this._checkTime();
  if (this.abort) return 0;

  // Before the transposition probe, which would
      otherwise hand back a score
  // for this position and hide the fact that the line
      has repeated it.
  if (this._isRepetition() || this.board.halfMoveClock
      >= 100) return this._drawScore();

  const hk = this.tt.computeHash(this.board);
  const [ttVal, ttMove] = this.tt.lookup(hk, depth,
      alpha, beta, ply);
  if (ttVal !== null) return ttVal;

  if (depth <= 0) return this._quiescence(alpha, beta);

  const gen = new MoveGenerator(this.board);
  const moves = gen.generateLegalMoves();
  const stm = this.board.sideToMove;
  const inCheck = gen.isInCheck(stm);
  if (!moves.length) return inCheck ? -MATE + ply :
      this._drawScore();

  // Null move: let the opponent move twice. If the
      reduced search still fails
  // high, a real move would too, so the node is cut
      without searching one.
  // Skipped in check (passing would be illegal),
      straight after another null
  // move, near mate scores, and with only king and
      pawns, where zugzwang is
  // common and passing is often the one thing the side
      to move cannot afford.
  if (this.nullMove && allowNull && !inCheck && depth
      >= this.nullMinDepth &&
      Math.abs(beta) < MATE_BOUND && this._hasPieces(
          stm)) {
    const score = -this._nullSearch(depth - 1 -
        NULL_MOVE_R, ply + 1, beta);
    if (this.abort) return 0;
    if (score >= beta) return beta;
  }

  const ordered = this._order(moves, ttMove);
  let bestScore = -Infinity, origAlpha = alpha,
      bestMove = null;
```

```
    for (const move of ordered) {
      if (this.abort) return 0;
      this.board.makeMove(move);
      this.repKeys.push(this.board.positionKey());
      const score = -this._negamax(depth - 1, ply + 1,
          -beta, -alpha, true);
      this.repKeys.pop();
      this.board.unmakeMove(move);
      if (score > bestScore) { bestScore = score;
          bestMove = move; }
      if (score > alpha) alpha = score;
      if (alpha >= beta) break;
    }
    if (this.abort) return 0;

    const flag = bestScore <= origAlpha ? TT_ALPHA :
        bestScore >= beta ? TT_BETA : TT_EXACT;
    this.tt.store(hk, depth, bestScore, flag, bestMove,
        ply);
    return bestScore;
  }

  // Passes the move and searches the opponent's reply
      with a null window at beta.
  // The pass is irreversible as far as repetition goes:
      the half-move clock is
  // zeroed for its duration, so no line below it can
      match a position above it.
  _nullSearch(depth, ply, beta) {
    const board = this.board;
    const { enPassant, halfMoveClock, sideToMove } =
        board;
    board.enPassant = null;
    board.halfMoveClock = 0;
    board.sideToMove = sideToMove === 'w' ? 'b' : 'w';
    this.repKeys.push(board.positionKey());
    const score = this._negamax(depth, ply, -beta, -beta
        + 1, false);
    this.repKeys.pop();
    board.sideToMove = sideToMove;
    board.halfMoveClock = halfMoveClock;
    board.enPassant = enPassant;
    return score;
  }
```

A.5 SOURCE CODE – QUIESCENCE SEARCH AND MOVE ORDERING (search.js)

```
  _quiescence(alpha, beta) {
    this._checkTime();
    if (this.abort) return 0;
    const raw = this.evaluator.evaluate(this.board);
    const standPat = this.board.sideToMove === 'w' ? raw
        : -raw;
    if (standPat >= beta) return beta;
    if (standPat > alpha) alpha = standPat;

    const gen = new MoveGenerator(this.board);
    const captures = gen.generateLegalMoves().filter(m =>
        m.pieceCaptured !== '.');
    for (const move of this._order(captures, 0)) {
      if (this.abort) return 0;
      this.board.makeMove(move);
      const score = -this._quiescence(-beta, -alpha);
      this.board.unmakeMove(move);
      if (score >= beta) return beta;
      if (score > alpha) alpha = score;
    }
    return alpha;
  }

  _order(moves, ttMove) {
    return moves.slice().sort((a, b) => scoreMove(b,
        ttMove) - scoreMove(a, ttMove));
  }

  // Any knight, bishop, rook or queen for `color`.
  _hasPieces(color) {
    const pieces = color === 'w' ? 'NBRQ' : 'nbrq';
    return this.board.squares.some(p => pieces.includes(
        p));
  }
}

// Hash move first, then MVV-LVA captures, then
    promotions. `ttMove` is the
// transposition table's packed best move, or 0.
export function scoreMove(move, ttMove) {
  let s = 0;
  if (ttMove && encodeMove(move) === ttMove) s = 10000;
  if (move.pieceCaptured !== '.') {
    s += 100 + (
        PIECE_ORDER[move.pieceCaptured.toLowerCase()] ||
        0) * 10 - (
        PIECE_ORDER[move.pieceMoved.toLowerCase()] || 0);
  }
  if (move.promotionPiece !== '.') s += 50;
```

```
  return s;
}
```

A.6 SOURCE CODE – ZOBRIST KEY COMPUTATION (transposition.js)

```
  computeHash(board) {
    let hi = 0, lo = 0;
    const squares = board.squares;
    for (let sq = 0; sq < 64; sq++) {
      const p = squares[sq];
      if (p === '.') continue;
      const k = PIECE_INDEX[p] * 64 + sq;
      hi ^= PIECE_KEYS.hi[k];
      lo ^= PIECE_KEYS.lo[k];
    }
    if (board.sideToMove === 'b') { hi ^= SIDE_KEY.hi[0];
        lo ^= SIDE_KEY.lo[0]; }
    const c = board.castlingRights;
    const ci = (c.K ? 1 : 0) | (c.Q ? 2 : 0) | (c.k ? 4 :
        0) | (c.q ? 8 : 0);
    hi ^= CASTLING_KEYS.hi[ci];
    lo ^= CASTLING_KEYS.lo[ci];
    if (board.enPassant !== null) {
      hi ^= EP_KEYS.hi[board.enPassant % 8];
      lo ^= EP_KEYS.lo[board.enPassant % 8];
    }
    return { hi, lo };
  }
```

A.7 DEPLOYMENT WORKFLOW (.github/workflows/deploy.yml)

```
name: Build & Deploy to Firebase Hosting

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:

jobs:
  build-and-deploy:
    name: Build & Deploy
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Test
        run: npm test

      - name: Build
        run: npm run build

      - name: Deploy to Firebase Hosting
        env:
          FIREBASE_TOKEN: ${{ secrets.FIREBASE_TOKEN }}
        run: npx firebase-tools deploy --only hosting
            --project foai-chess-engine --token
            "$FIREBASE_TOKEN"
```

A.8 REPRODUCING THE RESULTS

```
npm ci
npm test && npm run test:deep          # tests, perft on
    every published depth
npm install --no-save stockfish        # reference engine
    for matches
node bench/experiments.js --max-depth 7 --limit 30
node bench/tables.js
node bench/suite.js --engine foai --movetime 1000
node bench/tt-size.js
node bench/match.js --player foai --opponents sf:1500,
    sf:1700,sf:1900,sf:2100 \
    --movetime 1000 --rounds 16
```

A.9 SAMPLE ENGINE CONSOLE OUTPUT (RAPID GAME, ENGINE PLAYING BLACK)

```
d1 score +0.51 nodes 30 Qxd1+
d2 score +0.51 nodes 60 Qxd1+
d3 score +0.51 nodes 90 Qxd1+
d4 score +0.51 nodes 120 Qxd1+
d5 score +0.51 nodes 170 Qxd1+
d6 score +0.31 nodes 23,327 Qxd1+
d7 score +0.58 nodes 71,272 Qxd1+
Best move Qxd1+ · table 14% full · score −0.58 (White's
    view)
```